



**University of
Nottingham**

UK | CHINA | MALAYSIA

Final Report: Enhancing Financial Fraud Detection through Reinforcement Learning-based Oversampling

Submitted April 12, 2024, in partial fulfillment of
the conditions for the award of the degree
BSc Computer Science with Artificial Intelligence.

Patience Chew Yee CHEAH

20314828

scypc3@nottingham.edu.cn

Supervised by Dr Boon Giin LEE

School of Computer Science University of Nottingham Ningbo China

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Dr Boon Giin Lee, for his invaluable guidance and continuous feedback during my final year project. Without his support and motivation in my research journey, it would be impossible for me to complete this project. Special thanks to Yue Yang for offering me great help and support in this project as part of her PhD.

Abstract

Class imbalances are common in financial fraud datasets, resulting in the misclassification of numerous fraudulent cases by machine learning (ML) classifiers. Traditional oversampling methods tackle this imbalance by creating additional minority samples, but they often cannot optimize the target classifier’s performance. Consequently, the generated samples may contain noisy or uninformative data. Moreover, determining the optimal number of generated samples remains challenging. To overcome these challenges, this project proposed a Reinforcement Learning (RL)-based oversampling method, which integrates the target classifier into the process, to determine the appropriate minority samples that can improve the classifier’s performance. This method automatically discards uninformative samples and decides the suitable number of generated training samples, aiming to improve the classifier’s performance. Notably, this approach is compatible with any existing ML classifiers and oversampling methods. The proposed RL-based method outperformed existing oversampling techniques, highlighting the potential of RL in enhancing classifier performance and addressing class imbalance in financial fraud detection datasets.

Publications

Cheah, P. C. Y., Yang, Y., & Lee, B. G. (2023). Enhancing financial fraud detection through addressing class imbalance using hybrid SMOTE-GAN techniques. *International Journal of Financial Studies*, 11(3), 110. <https://doi.org/10.3390/ijfs11030110>

Cheah, P. C. Y., Yang, Y., & Lee, B. G. (2024). Reinforcement learning for optimizing conditional tabular generative adversarial networks hyperparameters in financial fraud detection. *2024 4th IEEE International Conference on Electrical, Computer, and Energy Technologies*. (Accepted)

Cheah, P. C. Y., Yang, Y., & Lee, B. G. (2024). Design of minority sample selection in imbalanced learning for financial fraud detection using reinforcement learning. *2024 1st IEEE System Council Student Event*. (Accepted)

Contents

Acknowledgments	i
Abstract	ii
Publications	iii
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background	1
1.2 Motivation	2
1.3 Aims and Objectives	3
1.4 Contributions	3
2 Literature Review	4
2.1 Oversampling Techniques	4
2.1.1 SMOTE-based Techniques	4
2.1.2 GAN-based Techniques	5
2.1.3 Hybrid SMOTE-GAN Techniques	6
2.2 Existing Oversampling Techniques' Performance	6
2.3 Reinforcement Learning Algorithms	7
2.3.1 Value-based Algorithms	7
2.3.2 Policy-based Algorithms	8
2.4 Reinforcement Learning in Imbalanced Learning	8
2.4.1 Data-level Approaches	9
2.4.2 Algorithm-level Approaches	9
2.4.3 Hybrid Approaches	10
3 Preliminary Work	11

3.1	Oversampling Methods Investigation	11
3.2	RL For CTGAN Hyperparameter Tuning	12
4	Methodologies	14
4.1	Data Preprocessing	14
4.2	Oversampling Methods Choices	15
4.3	Classifier Selection	15
4.4	Percentage of Resampled Minority Instances	16
4.5	RL Sample Selection Method	16
5	Results and Discussions	21
5.1	Resampled Minority Percentage Analysis	22
5.2	RL Sample Selection Method's Performance	29
5.3	Comparison with Existing Methods	36
5.4	The Best RL Sample Selection Model	37
6	Summary and Reflections	38
6.1	Project Management	38
6.2	Reflections	39
6.3	Conclusion and Future Work	40
	References	41
	Appendices	46
A	Proposed RL Method's Performance	46
B	Published Paper on IJFS	48
C	Accepted Paper For ICECET	66
D	SYSCON Acceptance Letter	73

List of Figures

5.1	Recall of Different Minority Percentage After Applying Four Oversampling Methods on South German Credit Dataset	23
5.2	Recall of Different Minority Percentage After Applying Four Oversampling Methods on African Crisis Dataset	24
5.3	Recall of Different Minority Percentage After Applying Four Oversampling Methods on Fake Bills Dataset	25
5.4	Recall of Different Minority Percentage After Applying Four Oversampling Methods on Default of Credit Card Clients Dataset	26
5.5	Recall of Different Minority Percentage After Applying Four Oversampling Methods on Credit Card Fraud Detection Dataset	27
5.6	Decision Tree On Possible Factors That Affect The Recall Improvement Generated by Python	28
5.7	Decision Tree on Possible Factors That Affects The Recall Improvements .	29
6.1	Updated Schedule and Milestones Completion	38
A.1	Training Mean Episode Reward on South German Credit Dataset	46
C.1	ICECET Acceptance Letter	66
D.1	SYSCON Acceptance Letter	73

List of Tables

4.1	Datasets Description	14
5.1	Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on South German Credit Dataset	23
5.2	Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on African Crisis Dataset	24
5.3	Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on Fake Bills Dataset	25
5.4	Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on Default of Credit Card Clients Dataset	26
5.5	Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on Credit Card Fraud Detection Dataset	27
5.6	Average Performance of XGBoost Before and After RL Sample Selection, With M=1.5	31
5.7	Average Performance of XGBoost Before and After RL Sample Selection, With M=2.0	32
5.8	Average Performance of XGBoost Before and After RL Sample Selection, With M=2.5	33
5.9	Average Performance of XGBoost Before and After RL Sample Selection, With M=3.0	34
5.10	Performance Improvement (%) Compared to Without Oversampling	35
5.11	Performance Improvement (%) Compared to Oversampling, Using M=2.0 .	36
A.1	Improvements (%) of The RL Sample Selection Method On Different M Values. Minor Improvement or Deterioration Between -1.0% and 1.0% (Exclusive) Were Omitted.	47

Chapter 1

Introduction

1.1 Background

Financial fraud, encompassing money laundering, insurance fraud, and credit card frauds, has caused substantial losses to financial institutions and the global economy [1]. Efforts have been made to develop automated fraud detection methods to alleviate the burden of manual identification. At an earlier stage, rule-based expert systems were employed for this purpose, but their inefficacy and costliness prompted a shift towards machine learning (ML) techniques [2]. Recent reviews [1, 2, 3] on financial fraud detection have highlighted various ML approaches like Random Forest, Support Vector Machines, Neural Networks, and Extreme Gradient Boosting (XGBoost). Although these techniques demonstrated satisfactory performance, the studies emphasized the challenge of imbalanced data.

Financial fraud datasets often exhibit class imbalance, with fraudulent proportion lower than the non-fraudulent ones. The benchmark Kaggle Credit Card Fraud Detection dataset [4], for instance, comprises a mere 0.17% of fraudulent instances. This imbalanced distribution poses a challenge in ML model training, as models tend to exhibit biased predictions towards the non-fraudulent class, resulting in a high misclassification rate of the fraudulent class [5]. Over the past years, the issue of class imbalance has gained increasing attention, as evidenced by the growing number of published papers addressing imbalanced learning techniques [6].

One prevalent method to tackle the class imbalance issue is oversampling, which involves generating new minority samples to increase the minority class representation in the dataset. Oversampling methods are compatible with existing learning methods [6], making them flexible and easy to use. Oversampling methods that generate new sample points, rather than merely duplicating existing data, improve the generalization [6]. Synthetic

Minority Oversampling Technique (SMOTE) [7] and Generative Adversarial Networks (GAN) [8] are two frequently used data generation techniques for rebalancing imbalanced datasets. In the financial fraud detection area, SMOTE has been the primary choice for addressing class imbalance by synthesizing new fraudulent instances [3]. Recently, GAN and its variants have also been utilized in the financial area for handling imbalanced datasets [9].

1.2 Motivation

When applying traditional oversampling methods, it is challenging to transform the data distribution effectively to optimize the ML model’s performance [6]. The absence of the target ML model during the oversampling process complicates the generation of informative minority samples that can aid the ML’s learning process. Consequently, oversampling methods may produce noisy and uninformative samples that could potentially degrade the performance of ML models. Results from a recent study [10] demonstrated that, on some datasets, ML classifiers exhibited deteriorated performance after applying oversampling methods.

Moreover, determining the optimal amount of generated samples to integrate into the training dataset poses a significant challenge. Earlier studies have shown that achieving an even distribution is not necessarily optimal, whether using randomly duplicated minority samples [11] or GAN-generated samples [12]. A study [9] raised questions regarding the optimum number of generated samples required to maximize ML performance.

Therefore, it is worth designing an oversampling method capable of optimizing generated samples based on the classifier’s performance while determining the optimal quantity of generated samples. Reinforcement learning (RL) is a learning paradigm that optimizes a strategy based on rewards or feedback. RL agents explore different strategies through trial and error to identify the most effective strategy for maximizing cumulative rewards [13]. The nature of RL renders it a potential candidate for optimizing generated samples based on the target classifier’s performance. By involving the target classifier in the oversampling process, it becomes possible to generate more effective minority samples for enhancing the classifier’s performance. Therefore, this project endeavours to design an RL-based oversampling method aimed at selecting generated minority samples for enhancing the ML classification performance on finance fraud datasets.

1.3 Aims and Objectives

This project aims to develop an RL-based sample selection technique to address the class imbalance problem in the finance fraud datasets. This technique seeks to optimize the sample selection strategy by maximizing the reward calculated based on the final classification performance when trained with the selected samples. The objectives to achieve this aim are:

1. To design an RL-based oversampling method to select suitable generated samples for enhancing classifiers' performance.
2. To investigate the relationship between the amount of generated samples and the resulting classification performance.
3. To evaluate the proposed RL-based method across multiple finance fraud datasets and compare it with the existing oversampling methods.

1.4 Contributions

This project made the following key contributions:

1. Proposed an RL-based sample selection method to mitigate the class imbalance problem, demonstrating performance improvements over existing oversampling methods on precisions, recalls, and F1-scores.
2. Analyzed the influence of the quantity of generated minority samples on classification performance.

Chapter 2

Literature Review

2.1 Oversampling Techniques

The imbalanced learning techniques can be classified into data-level, algorithm-level and hybrid approaches [5]. Oversampling is a data-level approach that adjusts the distribution of the training set by creating additional minority samples to achieve balance within the dataset. This data generation technique applies to any existing learning methods and can enhance generalization [6]. Compared to undersampling, which is another data-level method, oversampling typically yields superior results [14]. SMOTE and GAN are two commonly used oversampling techniques for augmenting the minority representation in datasets.

2.1.1 SMOTE-based Techniques

SMOTE is a standard oversampling method for imbalanced learning based on random interpolation between the nearest minority samples [7]. Initially, it randomly selects a subset of minority samples. For each selected sample point, SMOTE picks one of its k -nearest samples randomly and generates a sample on the line joining the two points. This simple approach has found wide application in finance fraud detection area [3]. However, SMOTE blindly synthesizes minority samples without considering the importance of individual minority data points and the majority instance's distribution, potentially producing uninformative samples.

To tackle the challenges, various studies have proposed variations of SMOTE. As of 2018, over 85 variations of SMOTE existed, including popular ones like SMOTE+ENN,

Borderline-SMOTE, ADASYN and Safe-Level SMOTE [15]. SMOTE+ENN integrates the Edited Nearest Neighbour (ENN) algorithm for data cleaning, removing misclassified samples from both classes after applying SMOTE [14]. Instead of sampling a random minority subset, Borderline-SMOTE specifically resamples the minority samples that lie close to the decision boundary to strengthen the boundary between minority and majority classes [16]. ADASYN generates additional samples around minority samples that are more challenging to learn [17]. Unlike SMOTE, which synthesizes additional samples randomly on the line between two minority instances, Safe-Level SMOTE restricts sample generation within a “safe-level” determined based on the nearest minority data points [18].

2.1.2 GAN-based Techniques

GANs [8] have found applications in creating realistic minority samples for imbalanced learning. A GAN consists of two components: a generator and a discriminator. The generator’s goal is to create realistic samples, while the discriminator’s aim is to differentiate between real and fake samples generated by the generator. Both components engage in a competitive process, continually improving each other’s performance. This technique has been adopted to create fraudulent samples for imbalanced finance fraud datasets and has demonstrated slightly higher recall rates than SMOTE [12].

Considering GAN’s original design for image generation and its instability in training, variations have been proposed. Balancing GAN (BAGAN) is trained with both minority and majority instances to learn their underlying patterns, aiming to generate minority class samples [19]. Conditional GAN (cGAN) incorporates additional conditions, such as the class labels, to train the generator to estimate the actual data distribution and synthesize minority samples [20]. Furthermore, the Conditional Tabular GAN (CTGAN) was specifically introduced to generate tabular data, such as financial data [21]. In finance datasets, various GANs were employed to generate minority samples, including the Wasserstein GAN (WGAN), which utilizes the Wasserstein loss function to stabilize the training process, and Conditional WGAN (cWGAN), which further combines conditional information in training [9]. Additionally, Unrolled GAN updates the generator several times before updating the discriminator, providing more feedback to the generator [22], while K-CGAN utilizes the Kilberg divergence loss function for training [23]. On finance fraud datasets, Unrolled GAN and K-CGAN outperformed SMOTE and its variations when measured using the F1-score [22, 23].

2.1.3 Hybrid SMOTE-GAN Techniques

Since SMOTE mainly relies on interpolating between the nearest minority samples, it focuses on local information, resulting in less diverse and realistic generated samples. Conversely, GAN necessitates a considerable amount of training data and poses challenges in training. To mitigate the limitations of each approach, recent studies have proposed hybrid methods combining SMOTE and GAN. SMOTified-GAN inputs SMOTE-generated minority samples for the GAN’s generator, instead of using random noises, to prevent generating samples from scratch [24]. ESMOTE-GAN is another hybrid approach that applies SMOTE and undersampling on several training subsets, followed by GAN training on the SMOTE-generated samples, to mitigate the impact of noise on GAN performance [25]. Their results indicate that ESMOTE-GAN outperforms SMOTE and GAN on financial datasets [25].

2.2 Existing Oversampling Techniques’ Performance

The performance improvements resulting from oversampling techniques on finance fraud datasets are often not substantial. For instance, Ye et al. [26] applied SMOTE to a financial statement fraud dataset, observing a 5.2% increase in precision compared to the scenario without oversampling. Despite a 1.3% decrease in the recall, the F1-score exhibited an improvement of 3.9% [26]. In contrast, Charitou et al. [27] applied SMOTE to a credit card fraud dataset, resulting in a 1.9% increase in recall but a 1.5% decrease in precision, leading to only a slight improvement of 0.2% in the F1-score [27]. In another study involving the South German Credit dataset, SMOTE led to a decrease of 1.9% in AUC [14].

In addition, Fiore et al. [12] applied GAN to the Kaggle Credit Card Fraud Detection dataset, achieving a 3.1% improvement in recall compared to the scenario without oversampling. Despite a decrease of 4.7% in precision, there was a slight improvement of 0.3% in the F1-score [12]. On the same dataset, SMOTified-GAN improved precision by 1%, albeit without an improvement in recall [24]. Additionally, Charitou et al. [27] managed to increase both precision and recall by 5.8% and 4.1%, respectively, using cGAN, resulting in a 0.5% improvement in the F1-score [27].

Overall, analysis of existing studies revealed that recall improvements resulting from resampling finance fraud datasets are typically below 5%, while precision can see higher increments, approaching almost 6%. Due to the precision-recall trade-off, F1-scores generally exhibit marginal improvements, usually within 1%. Therefore, achieving significant

enhancements with oversampling techniques in finance fraud datasets proves challenging. However, even slight improvement can be valuable for fraud detection systems, potentially leading to the identification of additional fraud cases and preventing substantial losses in the finance sector.

2.3 Reinforcement Learning Algorithms

RL is a ML method wherein an agent learns from its experience interacting with its environment to optimize a strategy aimed at maximizing cumulative rewards [28]. The standard RL formulation is typically in a Markov Decision Process (MDP), where the agent selects an action that alters its state and receives rewards or feedback depending on the resulting state's quality [13]. RL algorithms can be classified into two categories: value-based and policy-based.

2.3.1 Value-based Algorithms

Value-based RL aims to find a value function that estimates the expected sum of future rewards when in a given state. Typically, these algorithms approximate a Q-function, a type of value function that maps a state-action pair to a Q-value, which predicts the expected future rewards of selecting a specific action in a given state.

Traditional value-based algorithms, such as State-Action-Reward-State-Action (SARSA) [29] and Q-learning [30], estimate the Q-function by updating the Q-values iteratively based on the transitions being observed. Q-learning incorporates optimal Q-function of the subsequent state into its calculation, aiding in early convergence compared to SARSA [28]. However, these algorithms are mainly appropriate for environments characterized by a finite number of states and actions [13].

Due to the achievements of Deep Learning (DL), studies merged DL with RL to approximate the value function. These algorithms typically estimate the best action based on the approximated value function [28]. Deep Q-Network (DQN) [31] employs neural networks to estimate the Q-function, enabling it to support environments with unlimited states and discrete actions. Nevertheless, DQN often prioritizes actions with the highest Q-values regardless of their optimality [13]. To mitigate the overestimation, Double DQN splits the max operation in Q-function estimation into action selection and action evaluation, each approximated by a neural network [32]. Similarly, Dueling DQN separates the Q-function estimation into two, one for estimating the value of a state and another for estimating

the benefit of choosing a particular action over others, which is effective for states where the available actions exhibit minor differences [33].

2.3.2 Policy-based Algorithms

Policy-based RL algorithms differ from value-based methods in that they directly learn a function, or policy, that maps states to actions directly without estimating a value function. These algorithms are suitable for continuous state and action spaces due to their computational efficiency [28].

Policy gradient, a class of policy-based algorithms, update policy network’s parameters using gradient ascent to maximize the expected future reward [34]. However, traditional policy gradient methods can suffer from data inefficiency and instability. Proximal Policy Optimization (PPO) addresses these issues by incorporating a surrogate objective function, which constrains the policy update’s size, thereby avoiding significant policy changes to ensure more stable training [35].

Actor-critic algorithms fall within the policy-based category [28]. An actor-critic algorithm comprises two distinct networks: an actor to select an optimal action, and a critic to assess the action’s efficacy based on the resulting state and reward [13]. Deep Deterministic Policy Gradient (DDPG) employs deterministic policy gradients to expand the actor-critic framework to continuous action spaces [36]. Asynchronous Advantage Actor-Critic (A3C) improves upon traditional actor-critic methods by training multiple agents asynchronously on different environments in parallel, reducing computational requirements and data correlation [37]. RL algorithms often require careful tuning of hyperparameters and can suffer from data inefficiency. To address these issues, Soft Actor-Critic (SAC) maximizes entropy and expected future reward, leading to more stable performance [38].

2.4 Reinforcement Learning in Imbalanced Learning

Previous studies that employed RL for imbalanced learning typically formulated the problem within the context of MDP. They utilized a range of RL algorithms, including DQN and actor-critics, based on the problem formulation and algorithm appropriateness. These studies can be classified into three categories: data-level, algorithm-level, and hybrid-level approaches.

2.4.1 Data-level Approaches

At the data-level, RL has been adopted for two primary tasks: selecting training samples and generating minority samples. Initially, the Trainable Undersampling (TU) attempts to optimize the selection of training data using policy gradient, with rewards computed based on the classification performance [39]. TU outperformed traditional SMOTE-based oversampling methods. Expanding on TU’s success, the TRainable Undersampling with Self Training (TRUST) [40] labels the unlabeled samples with self-training and combines these samples with TU-selected ones for classifier training. TRUST exhibits superior performance in identifying minority samples compared to TU. Similarly, DiagSelect [41] employs the REINFORCE algorithm to select the optimal training subset based on classifier’s performance. On the other hand, for generating minority samples, an actor-critic-based method was proposed to directly synthesize minority samples, with simulated rewards derived from classifier performance [42]. This approach surpassed traditional distribution oversampling techniques across various datasets. Another approach, AutoSMOTE [43], enhances the traditional SMOTE technique by employing hierarchical RL to determine the optimal number of samples to synthesize around each minority point. The agent then decides the which neighbouring points of the minority data to choose, and determines the interpolation weight. This method demonstrates significant improvements over SMOTE-based and GAN-based oversampling methodologies.

2.4.2 Algorithm-level Approaches

At the algorithm-level, pioneering work such as DQNNimb [44] employs DQN to perform classification on imbalanced datasets. In DQNNimb, when the DQN agent identifies minority samples, it receives a higher reward computed based on the imbalance ratio of the dataset. By utilizing a similar reward formulation on Q-learning, a study found an optimal reward ratio that produced the highest accuracy [45]. In finance fraud detection, a study proposed a DQN method with a reward function modelled from the bank’s revenue model and the percentage of both misclassified and correctly identified fraud cases [46]. This approach surpassed most supervised learning methods, with the exception of XGBoost. Additionally, Double DQN was employed by awarding a maximum positive reward for correctly classifying fraudulent samples and a maximum negative reward for misclassification [47]. DDPG was adopted in the ResDPG framework for mechanical fault diagnosis with imbalanced dataset [48]. For imbalanced multi-class classification tasks, DQN was employed as a classifier after performing SMOTE and Gaussian Mixture Model for oversampling and undersampling respectively [49]. The performance of RL classifiers in imbalanced classification demonstrated in these studies is on par with benchmark

supervised learning approaches.

2.4.3 Hybrid Approaches

Some RL-related work in imbalanced learning do not fit neatly into either the data-level or algorithm-level categories. AESMOTE [50] represents a hybrid approach combining both data-level and algorithm-level to perform classification on imbalanced datasets. AESMOTE adopted two RL agents, one to select SMOTE-generated samples depending on the classification performance of the other agent that acts as the classifier. This method demonstrated superior performance compared to conventional undersampling methods. Moreover, another study [51] introduced an RL-based undersampling ensemble framework utilizing SAC to pick training subsets for individual classifiers, ultimately constructing an ensemble classifier. This approach exhibited superior performance over traditional hybrid imbalanced handling methods, particularly on datasets with significant overlapping.

Chapter 3

Preliminary Work

3.1 Oversampling Methods Investigation

This project’s preliminary work explored the performance of existing oversampling methods like SMOTE, GAN, and SMOTified-GAN on the Kaggle Credit Card Fraud Detection dataset. Existing literature revealed SMOTE’s and GAN’s weaknesses, leading to the innovation in hybridizing the two methods. SMOTified-GAN exhibited a higher F1-score than individual SMOTE and GAN [24]. Building upon this, our preliminary investigations designed two additional hybrids of SMOTE and GAN, published in a journal [52] (see Appendix B).

The first hybrid method, SMOTE+GAN, combines the samples generated using SMOTE and GAN, forming a balanced dataset with an equal representation of both types. The second method, GANified-SMOTE, applies SMOTE technique to the fraudulent samples generated by GAN, leveraging GAN’s ability to capture underlying fraudulent features and reducing the influence of noisy data. Further applying SMOTE reduces dependency on dominant GAN-generated patterns, mitigating overfitting issues.

In financial fraud detection, there often exists a trade-off between precision and recall [3]. While SMOTE+GAN achieved a high recall of approximately 90%, precision plummeted to around 30%, which is similar to SMOTE’s performance. GANified-SMOTE, though exhibiting relatively lower recall than SMOTE+GAN, consistently struck a balance between precision and recall, resulting in a high F1-score of about 85%. GANified-SMOTE yielded comparable performance as SMOTified-GAN.

The classifiers experimented with were Feed-forward Neural Network (FNN) and Convolutional Neural Network (CNN), both of which have outperformed traditional machine

learning methods in financial fraud detection. Additionally, a combination of the top-performing FNN and CNN models outperformed individual models.

Besides augmenting the fraudulent samples to 50% of the dataset, experiments also tripled the quantity of original fraudulent instances, inspired by prior research [12]. Results revealed that the proportion of fraudulent data in the training dataset significantly influenced final performance. This preliminary finding became an important consideration in the subsequent design of the RL-based oversampling method.

3.2 RL For CTGAN Hyperparameter Tuning

CTGAN, a GAN-based oversampling technique specifically designed for tabular data, was explored in the preliminary work. Experimentation revealed that a simple random search on CTGAN hyperparameters, including the generator’s and discriminator’s learning rates, could yield superior performance compared to default settings. Consequently, hyperparameter selection emerged as a crucial aspect in enhancing classification performance. Since RL has been successfully employed for hyperparameter optimization in prior research [53], the first attempt in applying RL in oversampling involved optimizing CTGAN hyperparameters, resulting in a paper accepted by a conference (see Appendix C).

In the designed RL framework, an agent observes the current CTGAN hyperparameter and determines whether to adjust its value or move to the next hyperparameter. At the end of each episode, the agent receives a reward based on the classifier’s recall and AUC after training with the CTGAN-generated samples using the chosen hyperparameter combinations. To address sparse and delayed rewards, the agent receives a small reward at each step based on the average quality of the current hyperparameter value derived from historical records. DQN, CEM, and SARSA were employed as RL agents in this work.

However, the performance of this method fell short of initial expectations regarding the agent’s ability to identify optimal CTGAN hyperparameter configurations. Although the reward function mainly considers the recall and AUC, these two metrics did not improve substantially on the test data. Conversely, precision and F1-score showed relatively higher enhancements, though overall improvements remained insignificant.

The unsatisfactory results may be due to an ineffective reward design and improper settings of the RL hyperparameters. Additionally, disparities in validation and testing sets distributions may have caused overfitting on the validation set. Even with optimized

hyperparameters, CTGAN may still produce noisy samples that impact final classification performance. This conclusion has led to the idea of designing an RL-based sample selection method, which subsequently became the main work of the FYP.

Chapter 4

Methodologies

4.1 Data Preprocessing

Table 4.1 provides details on five benchmark finance-fraud datasets utilized in this project. All datasets exhibit class imbalance, with the proportion of fraudulent instances being lower than that of non-fraudulent ones. This imbalance is particularly obvious in Datasets 2 and 5, as evidenced by their minority class percentages.

Dataset 4 contains several missing values, necessitating the removal of records with such values before further preprocessing. Standard data preprocessing procedures were applied to all five datasets, including the removal of unnecessary features, label encoding of categorical features, and min-max scaling for normalization. The datasets were then split into 70% training, 10% validation, and 20% testing while preserving the original class distributions.

Table 4.1: Datasets Description

No.	Dataset	Number of Features	Total Instances	Minority Percentage (%)
1.	South German Credit [54]	21	1000	30.00
2.	Africa Crisis [55]	14	1059	8.88
3.	Fake Bills [56]	7	1463	33.63
4.	Default of Credit Card Clients [57]	24	30000	22.12
5.	Credit Card Fraud Detection [4]	31	284807	0.18

4.2 Oversampling Methods Choices

Four oversampling methods were employed in this project to generate minority samples for utilization in the proposed RL sample selection method:

1. SMOTE [7]: SMOTE is recognized as the standard approach for addressing class imbalance, which is based on simple random interpolation between the nearest minority instances. The value of nearest neighbors (k value) was set to 5, following default settings.
2. GAN [8]: GAN was selected due to its growing popularity in handling imbalanced finance fraud datasets. The GAN employed in this project comprises a generator with four layers and a discriminator with three layers. Adam optimizer was utilized with 0.0005 learning rate for both the generator and discriminator. The networks underwent 150 training epochs, with a batch size of 128.
3. CTGAN [21]: CTGAN, a variation of GAN specifically proposed for tabular data, was employed for its suitability in handling finance fraud data. All hyperparameters for CTGAN were kept at their default values, except for the epoch number, which was reduced to 100 to accelerate the training process.
4. SMOTified-GAN [24]: SMOTified-GAN, a hybrid method incorporating SMOTE-generated samples into the GAN framework, was applied for oversampling. The SMOTE's and GAN's hyperparameters within the SMOTified-GAN framework were identical to those used for individual SMOTE and GAN methods.

4.3 Classifier Selection

Although deep learning demonstrates superior performance in text and image classification, tree ensemble learning algorithms continue to outperform them on tabular data [58, 59]. Due to the time constraints of this project, only XGBoost [60], a tree ensemble algorithm, was adopted as the classifier, given its relatively fast training speed compared to other gradient boosting methods. XGBoost is designed to handle sparse or missing data and can approximate error gradients with less computational power than traditional gradient boosting methods. The algorithm utilizes parallel tree constructions and caching mechanisms that fully optimize the hardware processors, thereby enhancing computational efficiency. XGBoost's capability to process large-scale datasets swiftly makes it

well-suited for the collected datasets, particularly Dataset 5, which constitutes approximately 0.3 million instances. The hyperparameters of XGBoost were configured to the default values provided by the library.

4.4 Percentage of Resampled Minority Instances

Imbalanced datasets are often oversampled to achieve perfect balance, where the number of majority and minority instances are equal. However, a study raised questions about the optimal number of minority samples that an oversampling method should generate to maximize classifier performance [9]. Thus, an experiment was conducted to study the influence of different percentage of generated samples on classification performance.

The four selected oversampling methods were applied to incrementally increase the fraudulent percentage to the nearest 10% intervals up to 70%. Percentages above 70% were not tested, as they would result in a severely imbalanced dataset with a higher proportion of fraudulent instances. All five collected datasets were experimented with to draw more general conclusions.

Results revealed that a perfectly balanced dataset often does not lead to optimal performance. However, there is no clear rule or trend that can describe the optimal minority proportion (further discussions in Section 5). Therefore, there is a need to automatically optimize the quantity of generated minority samples.

4.5 RL Sample Selection Method

Many existing oversampling methods exhibit the following issues:

1. Generation of noisy or uninformative samples.

Without integrating the target ML classifier in the oversampling process, it is challenging to modify the data distribution to optimize the classifier’s performance [6]. Thus, existing oversampling methods may generate uninformative samples that degrade the classifier’s performance. Moreover, the original minority data may contain noise, which is inevitably utilized to generate new minority samples, resulting in noisy generated samples. Additionally, as demonstrated in the preliminary work (Section 3.2), CTGAN may generate noisy samples that degrade the classifier’s performance, even with extensive hyperparameter optimization.

2. Inability to determine the optimal minority proportion.

Empirical results from the experiment in Section 4.4 show that the proportion of minority instances in a training set can affect the ML classifiers’ performance. A perfectly balanced dataset does not always yield the best performance. However, existing methods often require users to manually specify the number of generated samples. Typically the minority percentage is increased to 50%.

3. Lack of consideration for majority data.

Many oversampling techniques focus solely on the minority data distribution when generating new samples, neglecting the majority class. The four selected oversampling methods only utilize the original minority data as input, producing output based solely on minority information. Consequently, the generated samples may overlap with the majority instances, leading to an unclear class boundary.

Considering the above issues, this project designed a sample selection method aimed at choosing useful samples from a set of samples generated by existing oversampling methods based on the classifier’s performance. Ideally, the method will automatically filter out noisy and uninformative samples that could compromise classifier performance while dynamically determining the appropriate sample quantity for training. That means, if the number of minority samples is deemed sufficient, the method may discard any remaining samples.

Since there are no clear “rules” defining which generated samples should be selected, as well as the optimal sample quantity, the algorithm must self-explore different scenarios to find the relationships between these factors and the classifier performance. RL algorithms, which learn from experience by exploring different sequences of actions across diverse scenarios, are suitable for this sample selection task. The RL agent iteratively refines its sample selection strategy based on rewards or punishments issued based on the classifier’s performance.

To determine whether to include a generated sample in the training dataset, the agent requires a comprehensive understanding of the current distribution of the training set, encompassing both minority and majority distributions. Having this information helps the agent to make informed decisions. For instance, considering the location of the majority instances can prevent the selected samples from overlapping with the majority instances. Similarly, considering the minority instances’ locations ensures that the selected samples are within the boundaries of the minority class. While these considerations or strategies are purely based on intuitions, the RL agent is expected to identify optimal strategies which may differ from these intuitions.

The primary components of an RL framework comprise the agent, the observed environment, a range of possible actions, and the associated rewards. The detailed design of the RL framework is described below. The proposed approach is general to any oversampling method or classifier choice. Nevertheless, only the XGBoost classifier was investigated due to time constraints.

States

In each step, the agent observes the current state to decide on an action. Therefore, the state representation must provide sufficient information for the agent to make informed decisions. In this project, a state is represented using a 2D array with three rows and n columns, where n is the number of attributes in the dataset. Each row contains the following information:

- Row 1: The generated minority sample under consideration.

A sample is randomly selected from a set of generated minority samples. All samples are assumed to be normalized between -1 and 1. The sequence of the sample considered by the agent varies in each episode to prevent overfitting.

- Row 2: The average value of the nearest k minority instances of each attribute.

To compute this row, the algorithm first identifies the k nearest minority instances to the generated sample under consideration in the current training set. It then calculates the mean of each attribute across these k instances. The resulting values represent the average location of the nearest minority instances. This experiment used $k = 5$.

- Row 3: The average value of the nearest k majority instances of each attribute.

While the focus is on minority instances when generating new minority samples, it is important to consider information from majority instances as well. This row is computed similarly to Row 2, but using the k nearest majority instances. This experiment also used $k = 5$.

Actions

For each generated sample, the agent determines whether to retain or discard it. A retained sample is integrated into the training dataset with a minority class label, which immediately influences the subsequent state calculation. This approach enables the agent

to track the samples that have already been selected, preventing it from continuously choosing similar samples clustered around specific feature points.

Reward

A reward describes the quality of a state within an RL framework. A high positive reward encourages the agent to take the same action when encountering the same state in the subsequent iterations, whereas a negative reward or penalty discourages the agent. Thus, the formulation of an effective reward function is crucial to the agent’s learning process.

There is no direct way to assess the quality of a single generated sample, as the final classifier’s performance depends on the overall training samples, not a single sample. Hence, the reward function is formulated based on the classifier’s performance after the agent decides on all the generated samples. A reward is only assigned at the final step of each episode, while the intermediate steps receive zero reward. Although sparse and delayed rewards may pose challenges during training, no better intermediate reward function has come out thus far.

Evaluation of a classifier’s performance typically involves metrics like precision, recall, F1-score, and AUC. In finance fraud detection, emphasis is typically placed on maximizing recall, aiming to identify as many fraudulent instances as possible. To simplify the reward formulation, the reward function only considers the recall rate. If the recall improves after the sample selection, relative to the original training set, the agent will receive a positive reward. Otherwise, it will receive a negative reward. To normalize the reward between -1 to 1, the function was formulated in Equation 4.1. With this reward function, the agent is expected to learn the strategy to enhance the classifier’s recall rate.

$$Reward = \begin{cases} \frac{RC}{1-RC_{initial}} & \text{if } RC > 0 \\ \frac{RC}{RC_{initial}} & \text{if } RC \leq 0 \end{cases}, \text{ where } RC = RC_{final} - RC_{initial} \quad (4.1)$$

Agent

Stable-Baselines3 provides simple and reliable implementations of RL algorithms [61]. Among all the available RL algorithms, three that support discrete actions were employed: DQN, A2C and PPO. Since the observation space is a 2D non-image vector, the default policy network architecture flattened the observation space before passing it to a two-layer fully-connected network, with 64 neurons in each layer. The gamma value was set to 0.99

to prioritize future rewards, as rewards are only provided at the final step.

The RL networks were updated every $max_step \times 8$ steps, where max_step denotes the total amount of generated samples to be considered, which is also the number of steps per episode. Each agent underwent training for 320 episodes, with model checkpoints saved every 64 episodes. Hence, for each agent, five models were saved at different time steps for subsequent comparisons and the best model selection.

Chapter 5

Results and Discussions

The performance evaluation utilized commonly used metrics in finance fraud detection, namely Precision (PR), Recall (RC), F1-score (F1) and Area Under the Curve (AUC). The calculations for these metrics calculations are shown in Equations 5.1 - 5.3, where TP (True Positives) represents the number of correctly classified fraud cases, FP (False Positives) denotes non-fraudulent instances being misclassified as frauds, and FN (False Negatives) represents overlooked fraud cases. The AUC measures the area under the Receiver Operating Characteristics (ROC) curve, which is a plot of TP rate against FP rate. Thus, a higher AUC indicates a model's stronger ability to differentiate between two classes.

$$Precision = \frac{TP}{TP + FP} \quad (5.1)$$

$$Recall = \frac{TP}{TP + FN} \quad (5.2)$$

$$F_1score = \frac{2 \times PR \times RC}{PR + RC} \quad (5.3)$$

5.1 Resampled Minority Percentage Analysis

Studies often increase the number of fraudulent instances to form a perfectly balanced dataset. However, experimental results from this project (Tables 5.1 - 5.5 and Figures 5.1 - 5.5) demonstrate that such balance does not yield optimal performance. This aligns with the previous research finding [11] that experimented with duplicated minority samples. Nevertheless, there is no direct relationship between the minority percentage and the classifier’s performance. Different oversampling methods exhibit different performances when increasing the minority percentage.

Besides minority percentage, numerous other factors may influence classification performance, such as the number of original data, the number of features, the original minority percentage, and the oversampling method applied. To explore these potential factors further, a decision tree was trained on experiment results. For simplicity, only the recall improvement relative to no oversampling was analysed. The recall is considered more important due to the emphasis on identifying more frauds.

By default, Python Scikit-learn’s decision tree employs the Gini impurity criterion for splitting. The algorithm autonomously determines the branches or split points of the resulting decision tree, evaluating all potential thresholds or split points and selecting the one that minimizes the weighted sum of impurities for child nodes. The final tree is generated through recursive node splitting. The original decision tree figure generated with Python (Figure 5.6) is challenging to interpret due to the label names used. Thus, Figure 5.7 was manually drawn, with certain sub-trees pruned, to enhance general understanding.

In the decision tree, the minority percentage after oversampling (*Minority*) appears at deeper levels. Factors like the original minority percentage and the original number of instances are situated at shallower depths, highlighting their priority in the decision-making process. Hence, the optimal minority percentage after resampling and the choice of oversampling method are dependent on the dataset. Nevertheless, this decision tree only models some factors that affect the recall. The actual factors are potentially more complex, encompassing factors like noisy generated samples and overlapping between classes.

Based on the analysis, it becomes evident that each dataset necessitates a unique strategy to address imbalance issues, including the types and quantities of generated samples utilized. Therefore, the proposed RL sample selection method can be used to optimize such a strategy.

Table 5.1: Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on South German Credit Dataset

Oversampling	Minority %	PR	RC	F1	AUC
SMOTE	0.3	0.541	0.550	0.545	0.756
	0.4	0.489	0.567	0.525	0.743
	0.5	0.476	0.553	0.512	0.725
	0.6	0.505	0.570	0.535	0.739
	0.7	0.482	0.577	0.525	0.732
GAN	0.3	0.541	0.550	0.545	0.756
	0.4	0.515	0.557	0.535	0.743
	0.5	0.508	0.557	0.531	0.751
	0.6	0.500	0.560	0.528	0.738
	0.7	0.523	0.553	0.537	0.751
CTGAN	0.3	0.541	0.550	0.545	0.756
	0.4	0.492	0.547	0.518	0.745
	0.5	0.505	0.537	0.520	0.745
	0.6	0.489	0.537	0.511	0.746
	0.7	0.490	0.540	0.513	0.744
SMOTified-GAN	0.3	0.541	0.550	0.545	0.756
	0.4	0.507	0.567	0.535	0.749
	0.5	0.524	0.587	0.553	0.751
	0.6	0.513	0.567	0.538	0.744
	0.7	0.517	0.560	0.537	0.748

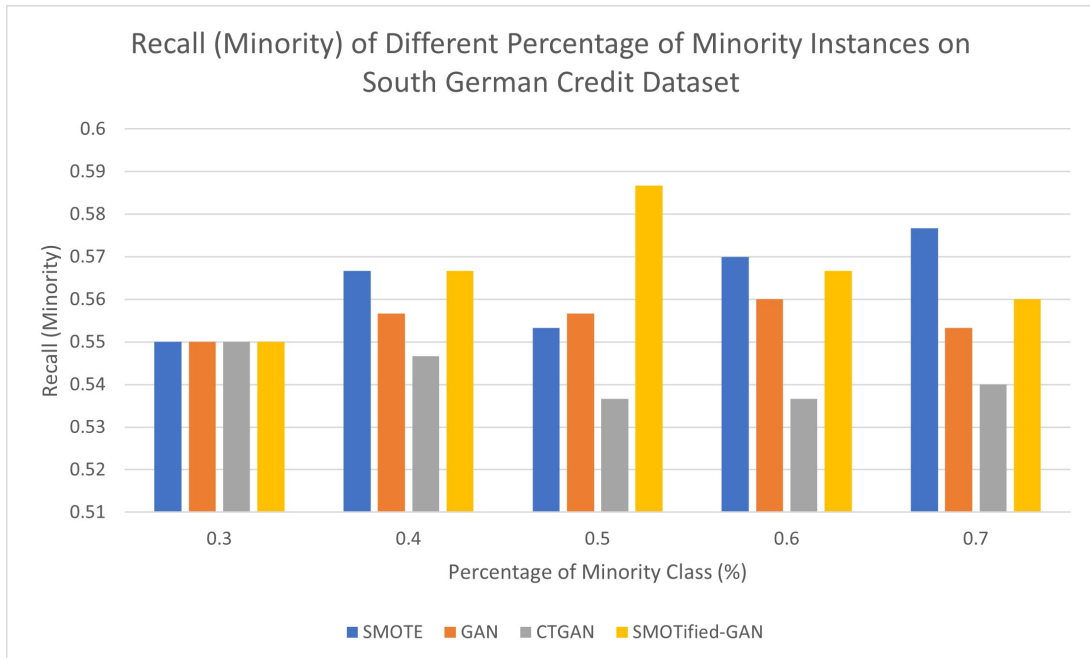


Figure 5.1: Recall of Different Minority Percentage After Applying Four Oversampling Methods on South German Credit Dataset

Table 5.2: Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on African Crisis Dataset

Oversampling	Minority %	PR	RC	F1	AUC
SMOTE	0.09	0.947	0.947	0.947	0.997
	0.1	0.949	0.989	0.969	0.997
	0.2	0.863	0.989	0.922	0.998
	0.3	0.834	1.000	0.909	0.998
	0.4	0.849	1.000	0.918	0.998
	0.5	0.856	1.000	0.922	0.998
	0.6	0.864	1.000	0.927	0.997
	0.7	0.856	1.000	0.922	0.998
GAN	0.09	0.947	0.947	0.947	0.997
	0.1	0.921	0.979	0.949	0.997
	0.2	0.941	1.000	0.969	0.998
	0.3	0.949	0.989	0.969	0.998
	0.4	0.950	1.000	0.974	0.997
	0.5	0.950	1.000	0.974	0.997
	0.6	0.950	1.000	0.974	0.997
	0.7	0.950	1.000	0.974	0.998
CTGAN	0.09	0.947	0.947	0.947	0.997
	0.1	0.921	0.979	0.949	0.998
	0.2	0.950	1.000	0.974	0.997
	0.3	0.950	1.000	0.974	0.999
	0.4	0.931	0.989	0.959	0.998
	0.5	0.950	1.000	0.974	0.997
	0.6	0.941	1.000	0.969	0.998
	0.7	0.950	1.000	0.974	0.998
SMOTified-GAN	0.09	0.947	0.947	0.947	0.997
	0.1	0.940	0.979	0.959	0.998
	0.2	0.950	1.000	0.974	0.997
	0.3	0.950	1.000	0.974	0.998
	0.4	0.932	1.000	0.965	0.998
	0.5	0.949	0.989	0.969	0.997
	0.6	0.941	1.000	0.969	0.998
	0.7	0.950	1.000	0.974	0.998

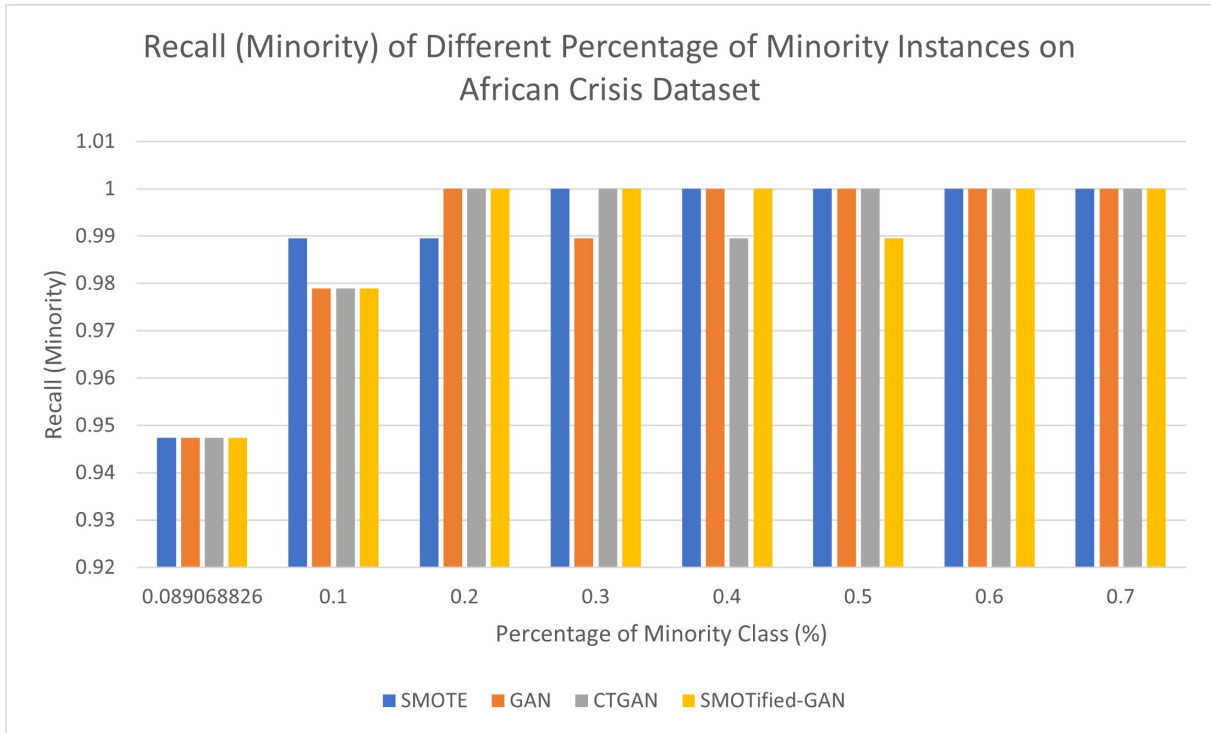


Figure 5.2: Recall of Different Minority Percentage After Applying Four Oversampling Methods on African Crisis Dataset

Table 5.3: Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on Fake Bills Dataset

Oversampling	Minority %	PR	RC	F1	AUC
SMOTE	0.34	0.989	0.949	0.969	0.997
	0.4	0.989	0.943	0.966	0.996
	0.5	0.989	0.949	0.969	0.996
	0.6	0.989	0.949	0.969	0.996
	0.7	0.990	0.954	0.971	0.997
GAN	0.34	0.989	0.949	0.969	0.997
	0.4	0.985	0.945	0.965	0.995
	0.5	0.974	0.956	0.964	0.992
	0.6	0.906	0.972	0.937	0.993
	0.7	0.899	0.966	0.931	0.992
CTGAN	0.34	0.989	0.949	0.969	0.997
	0.4	0.985	0.945	0.965	0.994
	0.5	0.967	0.954	0.960	0.994
	0.6	0.957	0.952	0.954	0.991
	0.7	0.917	0.964	0.940	0.993
SMOTified-GAN	0.34	0.989	0.949	0.969	0.997
	0.4	0.985	0.956	0.970	0.995
	0.5	0.965	0.945	0.955	0.994
	0.6	0.947	0.954	0.950	0.991
	0.7	0.926	0.958	0.940	0.994

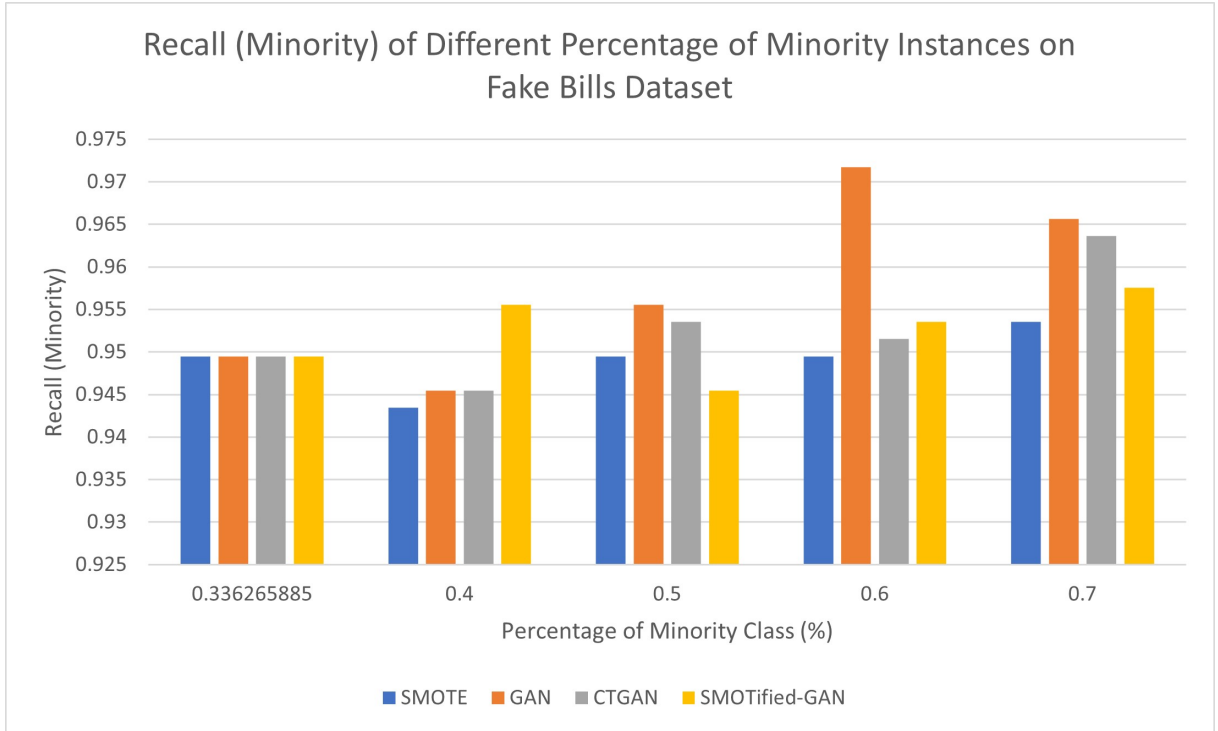


Figure 5.3: Recall of Different Minority Percentage After Applying Four Oversampling Methods on Fake Bills Dataset

Table 5.4: Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on Default of Credit Card Clients Dataset

Oversampling	Minority %	PR	RC	F1	AUC
SMOTE	0.22	0.632	0.368	0.465	0.758
	0.3	0.619	0.395	0.482	0.757
	0.4	0.577	0.405	0.476	0.749
	0.5	0.555	0.415	0.475	0.744
	0.6	0.544	0.427	0.479	0.741
	0.7	0.513	0.450	0.479	0.733
GAN	0.22	0.632	0.368	0.465	0.758
	0.3	0.637	0.366	0.465	0.762
	0.4	0.640	0.369	0.468	0.761
	0.5	0.638	0.361	0.461	0.765
	0.6	0.633	0.363	0.462	0.763
	0.7	0.633	0.358	0.458	0.763
CTGAN	0.22	0.632	0.368	0.465	0.758
	0.3	0.639	0.369	0.468	0.760
	0.4	0.642	0.369	0.468	0.764
	0.5	0.631	0.360	0.458	0.761
	0.6	0.635	0.367	0.465	0.764
	0.7	0.640	0.364	0.464	0.762
SMOTified-GAN	0.22	0.632	0.368	0.465	0.758
	0.3	0.634	0.367	0.465	0.761
	0.4	0.635	0.364	0.463	0.762
	0.5	0.634	0.364	0.462	0.761
	0.6	0.633	0.361	0.460	0.761
	0.7	0.643	0.364	0.465	0.764

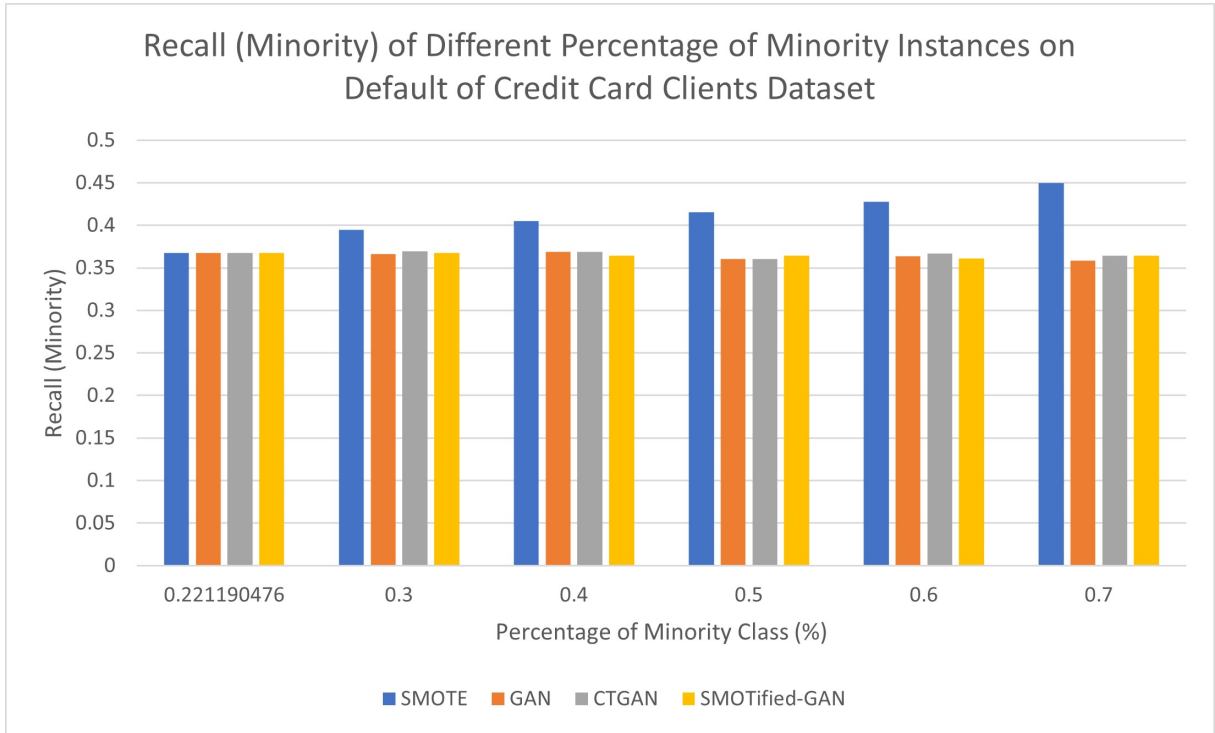


Figure 5.4: Recall of Different Minority Percentage After Applying Four Oversampling Methods on Default of Credit Card Clients Dataset

Table 5.5: Average Performance (5 Tests) of Four Oversampling Methods When Using Different Percentage of Minority Instances on Credit Card Fraud Detection Dataset

Oversampling	Minority %	PR	RC	F1	AUC
SMOTE	0.002	0.922	0.724	0.811	0.977
	0.1	0.859	0.771	0.813	0.982
	0.2	0.818	0.786	0.801	0.984
	0.3	0.806	0.804	0.805	0.984
	0.4	0.779	0.804	0.791	0.983
	0.5	0.753	0.804	0.778	0.984
	0.6	0.749	0.808	0.777	0.983
	0.7	0.729	0.810	0.767	0.983
GAN	0.002	0.922	0.724	0.811	0.977
	0.1	0.945	0.729	0.823	0.976
	0.2	0.933	0.739	0.825	0.979
	0.3	0.934	0.749	0.831	0.977
	0.4	0.928	0.733	0.819	0.978
	0.5	0.927	0.745	0.826	0.980
	0.6	0.919	0.737	0.818	0.978
	0.7	0.927	0.745	0.826	0.977
CTGAN	0.002	0.922	0.724	0.811	0.977
	0.1	0.940	0.741	0.829	0.980
	0.2	0.934	0.751	0.833	0.977
	0.3	0.926	0.737	0.820	0.979
	0.4	0.941	0.737	0.826	0.977
	0.5	0.936	0.743	0.828	0.977
	0.6	0.933	0.737	0.823	0.979
	0.7	0.936	0.743	0.828	0.976
SMOTified-GAN	0.002	0.922	0.724	0.811	0.977
	0.1	0.938	0.745	0.830	0.978
	0.2	0.926	0.739	0.822	0.975
	0.3	0.938	0.729	0.820	0.978
	0.4	0.940	0.739	0.827	0.976
	0.5	0.938	0.737	0.825	0.977
	0.6	0.941	0.716	0.813	0.979
	0.7	0.944	0.720	0.817	0.978

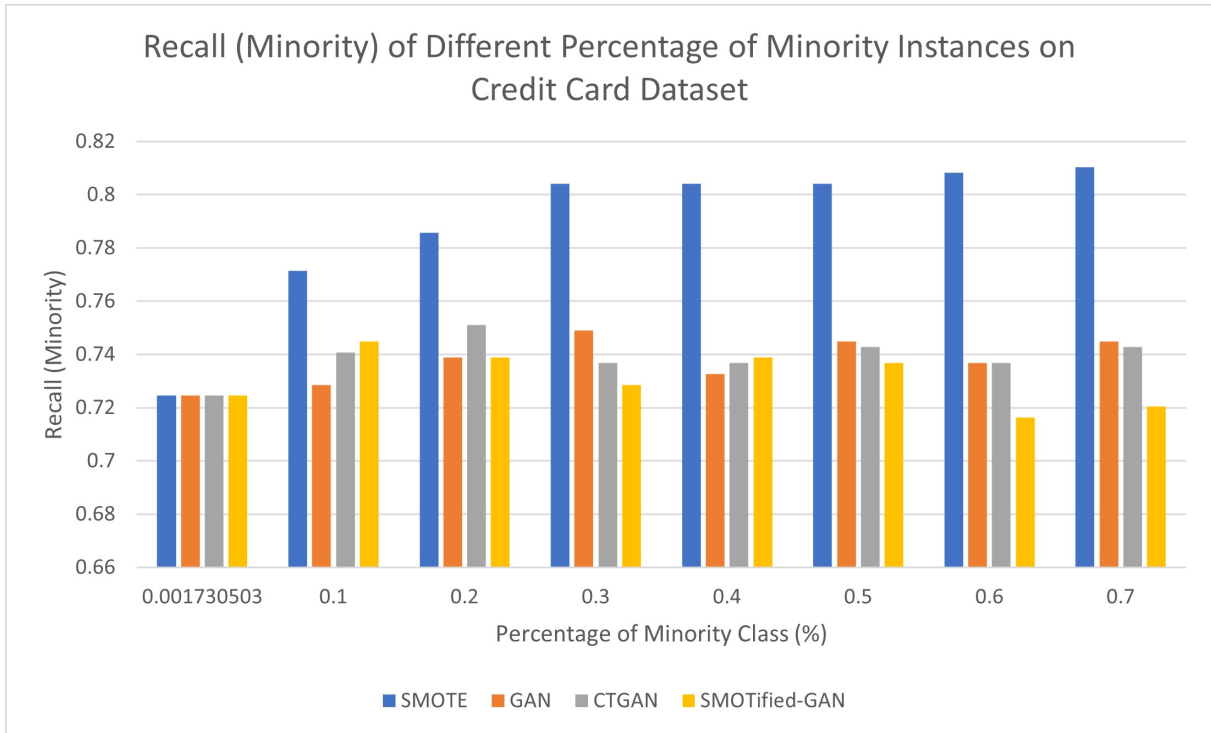


Figure 5.5: Recall of Different Minority Percentage After Applying Four Oversampling Methods on Credit Card Fraud Detection Dataset

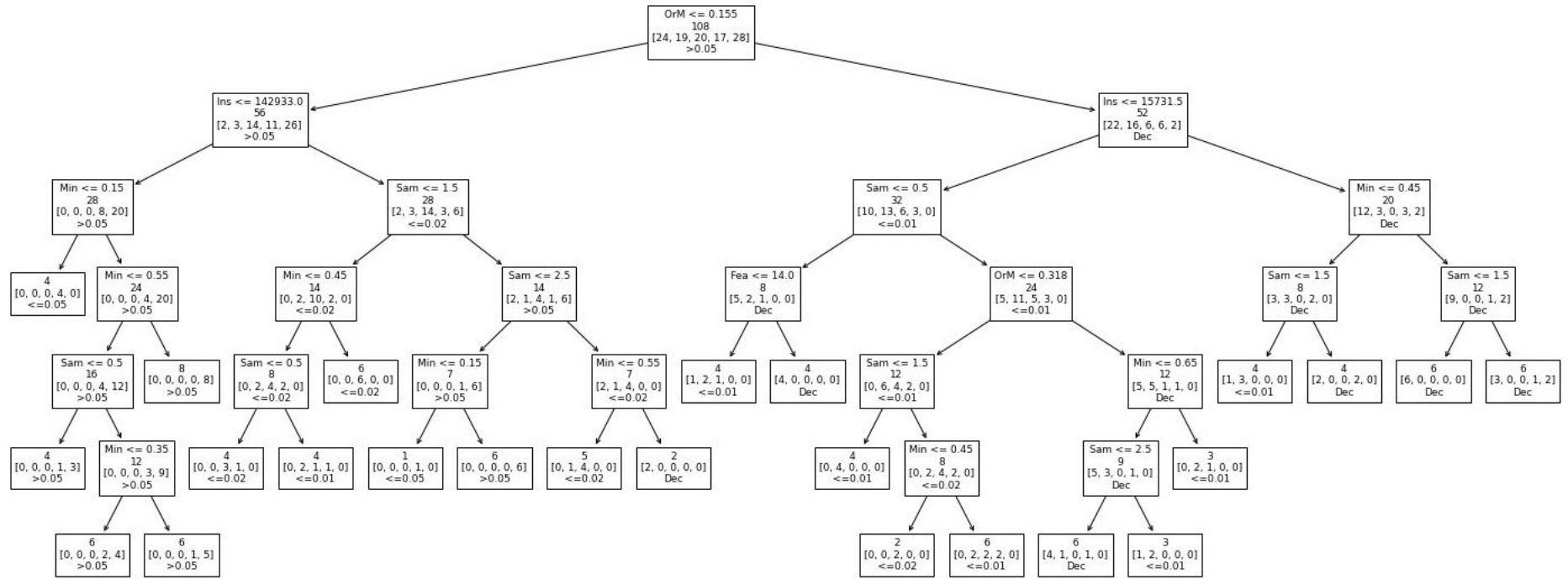


Figure 5.6: Decision Tree On Possible Factors That Affect The Recall Improvement Generated by Python

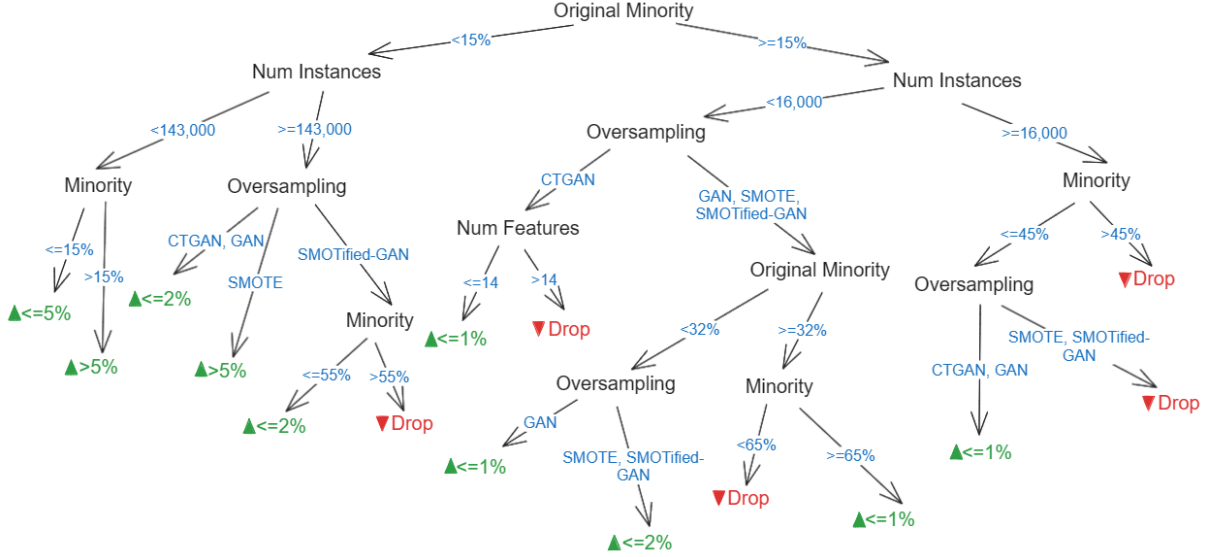


Figure 5.7: Decision Tree on Possible Factors That Affects The Recall Improvements

5.2 RL Sample Selection Method's Performance

The proposed RL method requires recomputing the state at each step, which can be time-consuming, particularly for larger datasets that require more training steps. Due to these constraints, the experiment evaluated the proposed method on only three of the five datasets, which contain fewer than 1500 instances. The two larger datasets, Datasets 4 and 5, were not tested. It is not worth spending long training time for only minor improvements.

For each dataset, four oversampling methods were employed to generate minority samples, and three distinct RL agents were trained on each set of generated samples. Each agent underwent training for 320 episodes, with five models saved at 64-episode intervals. The model that achieved the highest recall on the testing data among the five saved models was selected for subsequent testing and comparisons.

The number of generated samples, N , supplied for RL selection was originally designed as in Equation 5.4, where $Diff$ denotes the difference between the number of majority and minority instances, and M is a hyperparameter greater than 1. During the training process, M was set to 1.5 to provide the agent with more samples to choose from. However, due to the agent's setting, N needed to be an integer divisible by 8 to optimize the hardware utilization. Hence, Equation 5.5 was used in the actual implementation to slightly reduce the number of samples, ensuring that N is divisible by 8.

$$N = M \times Diff \quad (5.4)$$

$$N = \lfloor M \times Diff \rfloor - \lfloor (M \times Diff) \mod 8 \rfloor \quad (5.5)$$

All selected models underwent testing for five runs, and their average performances were detailed in Tables 5.6 - 5.9. During testing, the maximum number of samples allowed to be selected for training was capped at *Diff*. With $M = 1.5$, certain agents selected substantially fewer samples than *Diff*. This behaviour may be attributed to the agents deeming a smaller subset of samples sufficient to enhance classification performance. Another possible reason was the insufficient suitable samples supplied for selection. To avoid the insufficient samples, higher M values were tested to generate more samples for the agents to select.

Table 5.10 demonstrates the average improvements of the proposed method for different M values relative to the original performance without oversampling. Generally, recall exhibited an average increase of 1.9% compared to without oversampling. However, as a trade-off, precision showed an average decrease of 2.4%. resulting in a 0.4% decrease in F1-score. AUC also experienced an average decrease of 0.5%. The results show that the RL method slightly improves the recall, despite lowering precision. The results indicate that the RL method can marginally enhance recall, albeit with lower precision, suggesting that agents have learned a strategy to maximize reward based on recall improvements. However, the simple reward formulation does not consider other performance metrics, potentially contributing to declines in those measurements.

As the number of supplied generated samples (M) increases, agents tend to select more samples for training. One possible reason is that providing more generated samples increases the likelihood of obtaining higher-quality samples, enabling agents to have better choices. However, supplying excessive samples could degrade performance. When $M = 3.0$, the average recall improvement began to decline. $M = 2.0$ is considered the optimal value, as it yielded the highest recall improvement and the least decrease in precision and F1-score on average.

Table 5.6: Average Performance of XGBoost Before and After RL Sample Selection, With M=1.5

Dataset	Oversampling	RL	Selected Amount	PR	RC	F1	AUC
South German	SMOTE	-	280	0.488	0.553	0.519	0.740
		A2C	280	0.492	0.563	0.525	0.737
		DQN	275	0.479	0.553	0.513	0.742
		PPO	254	0.490	0.580	0.531	0.739
	GAN	-	280	0.516	0.573	0.542	0.753
		A2C	35	0.510	0.563	0.535	0.750
		DQN	277	0.499	0.573	0.533	0.741
		PPO	55	0.503	0.560	0.530	0.740
	CTGAN	-	280	0.507	0.553	0.529	0.748
		A2C	280	0.483	0.543	0.511	0.735
		DQN	280	0.493	0.560	0.524	0.744
		PPO	226	0.481	0.533	0.505	0.740
	SMOTified-GAN	-	280	0.521	0.550	0.535	0.742
		A2C	272	0.502	0.557	0.528	0.748
		DQN	274	0.516	0.557	0.535	0.749
		PPO	277	0.513	0.560	0.535	0.753
African Crisis	SMOTE	-	609	0.849	1.000	0.918	0.998
		A2C	609	0.849	1.000	0.918	0.997
		DQN	102	0.950	1.000	0.974	0.996
		PPO	609	0.856	1.000	0.922	0.998
	GAN	-	609	0.932	1.000	0.965	0.997
		A2C	584	0.950	1.000	0.974	0.997
		DQN	609	0.950	1.000	0.974	0.997
		PPO	563	0.941	1.000	0.969	0.997
	CTGAN	-	609	0.949	0.979	0.964	0.997
		A2C	0	0.947	0.947	0.947	0.997
		DQN	416	0.914	0.989	0.950	0.998
		PPO	507	0.950	1.000	0.974	0.997
	SMOTified-GAN	-	609	0.949	0.989	0.969	0.997
		A2C	609	0.950	1.000	0.974	0.997
		DQN	58	0.950	1.000	0.974	0.997
		PPO	376	0.941	1.000	0.969	0.997
Fake Bills	SMOTE	-	335	0.989	0.947	0.968	0.995
		A2C	335	0.989	0.949	0.969	0.995
		DQN	335	0.989	0.949	0.969	0.994
		PPO	109	0.989	0.947	0.968	0.996
	GAN	-	335	0.966	0.947	0.956	0.993
		A2C	334	0.954	0.956	0.955	0.993
		DQN	91	0.985	0.943	0.964	0.995
		PPO	58	0.989	0.949	0.969	0.996
	CTGAN	-	335	0.956	0.952	0.954	0.989
		A2C	335	0.954	0.949	0.952	0.990
		DQN	244	0.953	0.947	0.950	0.988
		PPO	335	0.953	0.949	0.951	0.990
	SMOTified-GAN	-	335	0.962	0.954	0.957	0.994
		A2C	170	0.983	0.949	0.966	0.996
		DQN	335	0.981	0.949	0.965	0.994
		PPO	306	0.966	0.956	0.961	0.995

Table 5.7: Average Performance of XGBoost Before and After RL Sample Selection, With M=2.0

Dataset	Oversampling	RL	Selected Amount	PR	RC	F1	AUC
South German	SMOTE	-	280	0.491	0.573	0.528	0.742
		A2C	280	0.488	0.570	0.526	0.737
		DQN	280	0.495	0.583	0.535	0.737
		PPO	280	0.488	0.563	0.523	0.735
	GAN	-	280	0.500	0.553	0.525	0.746
		A2C	32	0.522	0.573	0.546	0.747
		DQN	280	0.497	0.553	0.523	0.742
		PPO	78	0.497	0.553	0.524	0.733
	CTGAN	-	280	0.503	0.547	0.523	0.739
		A2C	280	0.518	0.570	0.543	0.750
		DQN	280	0.494	0.537	0.514	0.739
		PPO	270	0.503	0.567	0.533	0.753
	SMOTified-GAN	-	280	0.493	0.530	0.510	0.746
		A2C	262	0.492	0.540	0.515	0.750
		DQN	280	0.517	0.587	0.549	0.755
		PPO	280	0.519	0.573	0.545	0.745
African Crisis	SMOTE	-	609	0.851	1.000	0.919	0.998
		A2C	0	0.947	0.947	0.947	0.997
		DQN	609	0.897	1.000	0.946	0.998
		PPO	604	0.849	0.947	0.896	0.996
	GAN	-	609	0.950	1.000	0.974	0.997
		A2C	609	0.949	0.989	0.969	0.997
		DQN	609	0.949	0.979	0.964	0.997
		PPO	609	0.950	1.000	0.974	0.997
	CTGAN	-	609	0.950	1.000	0.974	0.997
		A2C	609	0.949	0.989	0.969	0.997
		DQN	609	0.950	1.000	0.974	0.997
		PPO	580	0.950	1.000	0.974	0.997
	SMOTified-GAN	-	609	0.949	0.979	0.964	0.997
		A2C	609	0.940	0.989	0.964	0.997
		DQN	80	0.950	1.000	0.974	0.997
		PPO	609	0.940	0.989	0.964	0.997
Fake Bills	SMOTE	-	335	0.989	0.949	0.969	0.996
		A2C	335	0.989	0.952	0.970	0.995
		DQN	335	0.989	0.949	0.969	0.995
		PPO	159	0.989	0.947	0.968	0.995
	GAN	-	335	0.951	0.958	0.954	0.993
		A2C	335	0.938	0.960	0.948	0.994
		DQN	52	0.989	0.945	0.967	0.996
		PPO	71	0.989	0.949	0.969	0.996
	CTGAN	-	335	0.967	0.954	0.960	0.992
		A2C	311	0.977	0.945	0.961	0.994
		DQN	275	0.963	0.954	0.958	0.990
		PPO	335	0.956	0.956	0.956	0.991
	SMOTified-GAN	-	335	0.948	0.956	0.951	0.996
		A2C	296	0.962	0.958	0.960	0.996
		DQN	321	0.978	0.954	0.965	0.995
		PPO	334	0.947	0.966	0.956	0.995

Table 5.8: Average Performance of XGBoost Before and After RL Sample Selection, With M=2.5

Dataset	Oversampling	RL	Selected Amount	PR	RC	F1	AUC
South German	SMOTE	-	280	0.499	0.570	0.532	0.737
		A2C	280	0.499	0.570	0.532	0.745
		DQN	280	0.490	0.563	0.524	0.742
		PPO	280	0.503	0.583	0.540	0.751
	GAN	-	280	0.511	0.553	0.531	0.749
		A2C	54	0.502	0.570	0.534	0.746
		DQN	280	0.512	0.570	0.539	0.750
		PPO	129	0.515	0.573	0.542	0.741
	CTGAN	-	280	0.512	0.593	0.549	0.747
		A2C	280	0.490	0.540	0.513	0.740
		DQN	280	0.501	0.553	0.525	0.741
		PPO	277	0.498	0.543	0.519	0.742
	SMOTified-GAN	-	280	0.494	0.547	0.519	0.743
		A2C	280	0.497	0.560	0.526	0.743
		DQN	280	0.520	0.567	0.542	0.750
		PPO	280	0.513	0.533	0.523	0.745
African Crisis	SMOTE	-	609	0.834	1.000	0.909	0.997
		A2C	0	0.947	0.947	0.947	0.997
		DQN	609	0.888	1.000	0.941	0.998
		PPO	609	0.858	0.958	0.905	0.996
	GAN	-	609	0.949	0.989	0.969	0.997
		A2C	609	0.940	0.989	0.964	0.997
		DQN	600	0.949	0.989	0.969	0.997
		PPO	609	0.950	1.000	0.974	0.997
	CTGAN	-	609	0.950	1.000	0.974	0.997
		A2C	609	0.950	1.000	0.974	0.997
		DQN	609	0.941	1.000	0.969	0.997
		PPO	609	0.931	0.989	0.959	0.998
	SMOTified-GAN	-	609	0.950	1.000	0.974	0.998
		A2C	609	0.950	1.000	0.974	0.998
		DQN	59	0.940	0.989	0.964	0.997
		PPO	606	0.950	1.000	0.974	0.998
Fake Bills	SMOTE	-	335	0.989	0.947	0.968	0.996
		A2C	335	0.989	0.949	0.969	0.996
		DQN	335	0.989	0.949	0.969	0.995
		PPO	190	0.989	0.949	0.969	0.995
	GAN	-	335	0.975	0.954	0.964	0.991
		A2C	335	0.970	0.962	0.966	0.992
		DQN	92	0.989	0.949	0.969	0.996
		PPO	130	0.989	0.952	0.970	0.995
	CTGAN	-	335	0.955	0.958	0.956	0.995
		A2C	335	0.960	0.956	0.958	0.994
		DQN	320	0.938	0.954	0.945	0.993
		PPO	335	0.960	0.947	0.953	0.994
	SMOTified-GAN	-	335	0.960	0.958	0.959	0.995
		A2C	303	0.964	0.964	0.964	0.996
		DQN	335	0.964	0.958	0.961	0.995
		PPO	335	0.960	0.966	0.963	0.996

Table 5.9: Average Performance of XGBoost Before and After RL Sample Selection, With M=3.0

Dataset	Oversampling	RL	Selected Amount	PR	RC	F1	AUC
South German	SMOTE	-	280	0.491	0.590	0.536	0.747
		A2C	280	0.508	0.587	0.544	0.739
		DQN	280	0.489	0.573	0.528	0.736
		PPO	280	0.492	0.550	0.519	0.742
	GAN	-	280	0.485	0.547	0.514	0.740
		A2C	70	0.513	0.580	0.544	0.746
		DQN	280	0.523	0.567	0.543	0.746
		PPO	197	0.510	0.563	0.535	0.742
	CTGAN	-	280	0.495	0.537	0.515	0.744
		A2C	280	0.495	0.563	0.527	0.743
		DQN	280	0.494	0.543	0.517	0.749
		PPO	280	0.502	0.560	0.529	0.743
	SMOTified-GAN	-	280	0.529	0.573	0.550	0.748
		A2C	280	0.502	0.573	0.535	0.740
		DQN	280	0.517	0.557	0.536	0.746
		PPO	280	0.496	0.547	0.520	0.749
African Crisis	SMOTE	-	609	0.849	1.000	0.918	0.998
		A2C	0	0.947	0.947	0.947	0.997
		DQN	609	0.905	1.000	0.950	0.998
		PPO	609	0.849	0.947	0.896	0.995
	GAN	-	609	0.949	0.989	0.969	0.997
		A2C	609	0.939	0.979	0.959	0.997
		DQN	609	0.939	0.979	0.959	0.996
		PPO	609	0.950	1.000	0.974	0.997
	CTGAN	-	609	0.950	1.000	0.974	0.997
		A2C	609	0.941	1.000	0.969	0.997
		DQN	609	0.950	1.000	0.974	0.997
		PPO	566	0.941	1.000	0.969	0.997
	SMOTified-GAN	-	609	0.949	0.989	0.969	0.997
		A2C	609	0.949	0.989	0.969	0.997
		DQN	108	0.940	0.979	0.959	0.997
		PPO	609	0.949	0.989	0.969	0.997
Fake Bills	SMOTE	-	335	0.989	0.952	0.970	0.996
		A2C	335	0.989	0.949	0.969	0.996
		DQN	335	0.989	0.949	0.969	0.996
		PPO	244	0.989	0.949	0.969	0.996
	GAN	-	335	0.980	0.954	0.966	0.996
		A2C	266	0.973	0.949	0.961	0.995
		DQN	190	0.989	0.947	0.968	0.996
		PPO	149	0.989	0.949	0.969	0.995
	CTGAN	-	335	0.965	0.947	0.956	0.992
		A2C	319	0.951	0.949	0.950	0.990
		DQN	327	0.945	0.956	0.949	0.991
		PPO	335	0.968	0.949	0.958	0.991
	SMOTified-GAN	-	335	0.958	0.952	0.954	0.994
		A2C	289	0.954	0.960	0.957	0.997
		DQN	335	0.954	0.952	0.953	0.995
		PPO	335	0.951	0.962	0.956	0.995

Table 5.10: Performance Improvement (%) Compared to Without Oversampling

M	Oversampling	RL	PR	RC	F1	AUC
1.5	SMOTE	A2C	-4.9	2.2	-1.6	-0.7
		DQN	-2.0	1.9	-0.1	-0.6
		PPO	-4.7	2.7	-1.3	-0.6
	GAN	A2C	-2.1	2.4	0.1	-0.3
		DQN	-1.4	2.4	0.4	-0.6
		PPO	-1.5	2.1	0.2	-0.6
	CTGAN	A2C	-3.1	-0.2	-1.7	-0.9
		DQN	-3.9	1.7	-1.3	-0.7
		PPO	-3.1	1.2	-1.0	-0.8
	SMOTified-GAN	A2C	-1.4	2.0	0.2	-0.3
		DQN	-1.0	2.0	0.5	-0.3
		PPO	-1.9	2.3	0.1	-0.1
	AVERAGE		-2.6	1.9	-0.5	-0.5
	SMOTE	A2C	-1.7	0.8	-0.6	-0.7
		DQN	-3.2	2.9	-0.4	-0.7
		PPO	-5.0	0.4	-2.5	-0.8
2.0	GAN	A2C	-2.2	2.5	0.1	-0.4
		DQN	-1.4	1.1	-0.2	-0.5
		PPO	-1.3	1.9	0.2	-0.8
	CTGAN	A2C	-1.1	2.0	0.4	-0.3
		DQN	-2.3	1.5	-0.5	-0.8
		PPO	-2.3	2.5	0.1	-0.3
	SMOTified-GAN	A2C	-2.7	1.4	-0.7	-0.2
		DQN	-1.1	3.1	0.9	-0.1
		PPO	-2.3	2.7	0.1	-0.4
	AVERAGE		-2.2	1.9	-0.3	-0.5
	SMOTE	A2C	-1.4	0.7	-0.4	-0.4
		DQN	-3.6	2.2	-0.9	-0.5
		PPO	-4.2	1.5	-1.6	-0.2
	GAN	A2C	-2.2	2.5	0.1	-0.5
		DQN	-0.9	2.1	0.5	-0.2
		PPO	-0.8	2.6	0.9	-0.6
2.5	CTGAN	A2C	-2.6	1.7	-0.5	-0.7
		DQN	-3.2	2.0	-0.7	-0.7
		PPO	-3.0	1.1	-1.0	-0.5
	SMOTified-GAN	A2C	-2.2	2.3	0.2	-0.3
		DQN	-1.7	2.3	0.2	-0.3
		PPO	-1.8	1.8	0.0	-0.4
	AVERAGE		-2.3	1.9	-0.3	-0.4
	SMOTE	A2C	-1.1	1.3	0.0	-0.6
		DQN	-3.1	2.6	-0.5	-0.7
		PPO	-4.9	0.0	-2.6	-0.6
	GAN	A2C	-1.7	2.1	0.1	-0.4
		DQN	-0.8	1.6	0.3	-0.4
		PPO	-0.9	2.2	0.6	-0.5
3.0	CTGAN	A2C	-3.0	2.2	-0.5	-0.7
		DQN	-2.9	1.8	-0.7	-0.4
		PPO	-2.2	2.1	-0.1	-0.6
	SMOTified-GAN	A2C	-2.4	2.5	0.0	-0.6
		DQN	-2.2	1.4	-0.5	-0.4
		PPO	-2.7	1.7	-0.5	-0.3
	AVERAGE		-2.3	1.8	-0.4	-0.5

5.3 Comparison with Existing Methods

Previous analysis has identified the optimal M value as 2.0. Thus, this project further compares the proposed RL sample selection method using $M = 2.0$, with the existing oversampling methods to evaluate its effectiveness in selecting appropriate training samples. Table 5.11 presents the performance improvement after applying sample selection across different oversampling methods.

Table 5.11: Performance Improvement (%) Compared to Oversampling, Using M=2.0

Oversampling	RL	PR	RC	F1	AUC
SMOTE	A2C	3.1	-1.8	0.9	-0.2
	DQN	1.7	0.3	1.1	-0.2
	PPO	-0.1	-2.2	-1.0	-0.3
GAN	A2C	0.3	0.4	0.3	0.0
	DQN	1.1	-1.1	0.0	-0.1
	PPO	1.2	-0.3	0.4	-0.4
CTGAN	A2C	0.8	0.2	0.5	0.4
	DQN	-0.4	-0.3	-0.4	0.0
	PPO	-0.4	0.7	0.2	0.4
SMOTified-GAN	A2C	0.2	0.8	0.5	0.1
	DQN	1.8	2.5	2.1	0.2
	PPO	0.6	2.1	1.3	-0.1
AVERAGE		0.8	0.1	0.5	0.0

All average performance metrics exhibit minor positive improvements, except for AUC, which remains unchanged. The precision shows greater improvement than the recall despite the RL method being designed to enhance the recall. This may be attributed to the RL reward formulation, which evaluates recall improvement relative to using the original training dataset without oversampling. Consequently, when comparing the RL method to the existing oversampling method, the recall improvement became less pronounced. Nevertheless, the average F1-score improvement is positive, indicating that the sample selection method can slightly outperform existing oversampling methods.

Although the experiment focused on four existing oversampling methods - SMOTE, GAN, CTGAN and SMOTified-GAN - the RL sample selection approach can be applied to other oversampling methods to select suitable generated samples for training. This highlights the generality of the proposed method, which can improve the performance of any existing oversampling methods. However, the percentage of improvement depends on the dataset and oversampling method employed. Applying RL after SMOTified-GAN yields the greatest recall improvement compared to the other techniques. Despite SMOTified-GAN originally exhibiting relatively lower recall than other methods, the proposed RL method enables its recall to surpass the others (see Table 5.7), resulting in substantial

recall improvement. For instance, on the South German Credit dataset, SMOTified-GAN with DQN achieved an average recall of 58.7%, the highest among the methods. On the Fake Bills dataset, SMOTified-GAN with PPO achieved the highest recall of 96.6%.

Despite the slight improvement relative to the existing methods, the proposed RL method requires a long training time. To calculate the reward at the end of each episode, an XGBoost classifier needs to be trained on the selected samples, which is time-consuming, particularly for large datasets. Additionally, if the original minority percentage is extremely low, the number of supplied generated samples will be high, resulting in more steps per episode. Since computing the state in each step involves determining the nearest neighbours, this process is computationally intensive. Due to the expensive training time, this method may not be practical for large datasets. Nevertheless, for small datasets, investing little time to achieve slight performance improvements remains reasonable. For example, the South German Credit dataset, which comprises 1,000 instances, took approximately six minutes to train for 320 episodes. The Default of Credit Card Clients dataset, which comprises 30,000 instances, requires more than five hours of training. The training time varies depending on the RL method used, with DQN typically requiring less time.

5.4 The Best RL Sample Selection Model

When compared to the performance without oversampling, SMOTified-GAN with DQN on $M = 2.0$ demonstrated the highest recall improvement of 3.1% and F1-score improvement 0.9%, while exhibiting the least drop in AUC. When compared to the existing oversampling methods, SMOTified-GAN DQN also achieved the highest improvements on recall and F1-score of 2.5% and 2.1%, respectively. Its precision improved by 1.8%, and the AUC slightly improved by 0.2%. Hence, based on the performance, SMOTified-GAN DQN can be considered the best model.

Chapter 6

Summary and Reflections

6.1 Project Management

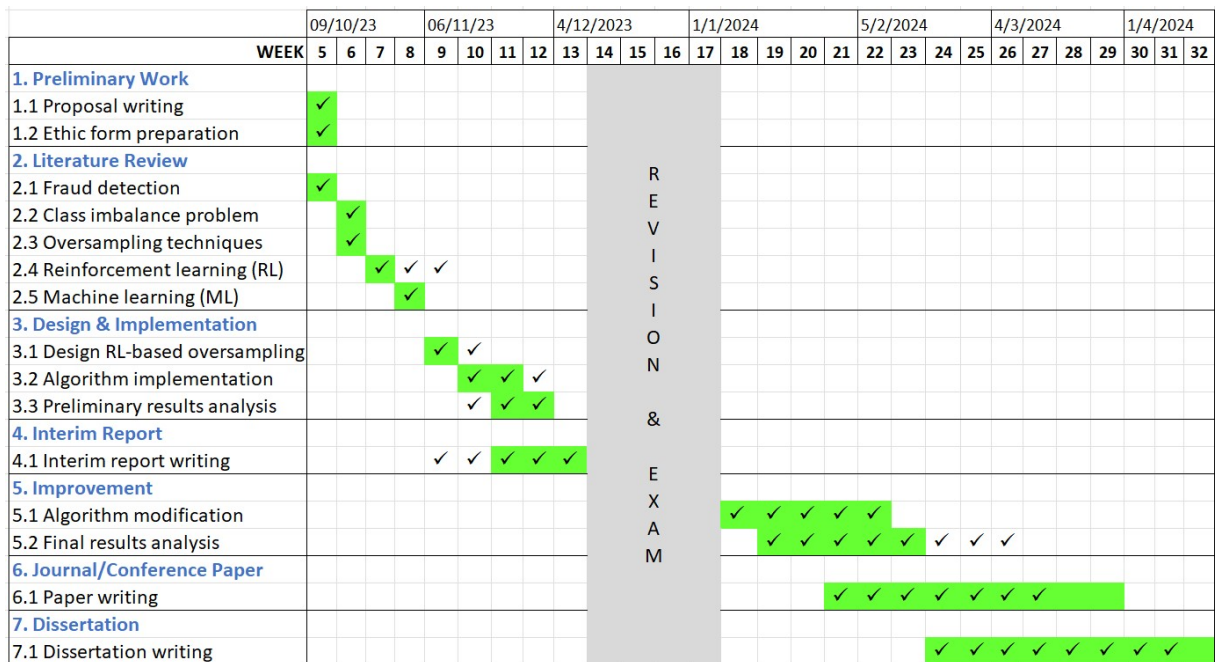


Figure 6.1: Updated Schedule and Milestones Completion

The Gantt Chart (Figure 6.1) illustrates the updated version of project schedule with completed milestones denoted by check marks. In general, the project was effectively managed, and all the scheduled tasks were successfully accomplished. A significant portion of preliminary work, including background studies, preliminary design and initial implementation, have been started during the summer vacation. As a result, less time was allocated to literature review and algorithm design during the autumn semester. Instead, the focus was on the implementation and refinement of the proposed method, as

well as performance analysis.

To fulfil the initial plan, two papers related to RL oversampling methods were authored during the project timeline. One paper detailed the CTGAN hyperparameter optimization method (considered preliminary work), while the other described the proposed sample selection method. The former has been accepted for presentation at the International Conference on Electrical, Computer, and Energy Technologies (ICECET), while the latter has been accepted for presentation at the 2024 1st IEEE Systems Council Student Event (SYSCON).

6.2 Reflections

Despite facing challenges, this project successfully addressed them and implemented the proposed RL sample selection method. Overcoming these obstacles and identifying mistakes allowed me to gain valuable experience in conducting research projects.

At the beginning of this project, the unfamiliarity with RL made it hard to understand the research papers. To overcome this issue, I studied online articles and watched videos that explained the RL concepts. Throughout the project, I continually reinforced my understanding by reading more papers and conducting experiments.

Reproducing the existing works is challenging. The initial plan was to use the latest works as a foundation to build a new method. Unfortunately, most research papers are not open source. As a beginner in RL, it was hard to implement the latest and most advanced work without access to source code. Consequently, the project focused on implementing basic RL techniques rather than the latest advancements.

Deciding the most appropriate software libraries is difficult. Although many Python RL libraries are available, choosing the most suitable one is not straightforward. Initially, this project worked on Keras-RL for its flexibility and ease of use. However, Keras-RL does not include advanced RL algorithms like PPO and actor-critics. Hence, in the middle of this project, Stable-Baseline3 was employed as it offers more advanced algorithms. The change in library selection caused compatibility issues. Luckily, these issues were addressed by modifying the initial code. This emphasizes the significance of making right decision at the start of a project to prevent later inconvenience.

The biggest challenge is the design and implementation of the RL oversampling method. The mean episode reward when training the RL agents remains negative, possibly caused by the improper reward function design. Even after several efforts to redesign the reward

function, there was no improvement in the training performance. Additionally, the plan to customize the feature extractor and policy network using convolutional layers faced implementation issues and was ultimately not implemented. Despite these challenges, the proposed method improves the average performance of pure oversampling.

Much like an RL agent learns from experience interacting with the environment, I gained research experience throughout this project. I learn from achievements and mistakes like the RL agent learns from rewards and punishments. This experience will help me to make better decisions when faced with similar situations in my future research journey.

6.3 Conclusion and Future Work

This project addressed the class imbalance problem within financial fraud datasets by introducing an RL-based method to find the optimal strategy for selecting appropriate generated training samples. The empirical findings revealed the influence of the number of generated minority samples on the final classification performance, highlighting the challenge of determining an optimal quantity. The proposed method can select suitable samples while determining the sample quantity based on the target ML classifier performance. In addition, the proposed method is general as it supports generated samples from any oversampling methods and can be used regardless of classifier choice. Despite a slight decrease in precision, the proposed method showed an average recall improvement of 1.9%. Notably, compared to existing oversampling techniques, the proposed RL method showed enhancement of 0.8% on precision, 0.1% on recall, and 0.5% on F1-score. These improvements demonstrate the potential of RL in enhancing classifier performance through sample selection.

Future work could focus on refining the reward calculation within the RL framework to address the challenge of sparse and delayed rewards, particularly as the current design supplies rewards only at the end of each episode. Integrating other performance metrics like precision and F1-score into the reward calculation could provide more valuable feedback to the agent. Additionally, the current reward calculation requires retraining the classifier, which is time-consuming, particularly on large datasets. This makes the proposed method impractical for real-world applications, which typically involve large datasets. Therefore, exploring novel techniques to evaluate or estimate classifier performance on the resampled dataset could be future research. Besides modifications to the current design, investigating the potential of applying RL at the algorithm-level rather than the data-level in addressing class imbalance is a promising direction for further exploration.

References

- [1] W. Hilal, S. A. Gadsden, and J. Yawney, “Financial fraud: A review of anomaly detection techniques and recent advances,” *Expert Systems with Applications*, vol. 193, p. 116429, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417421017164>
- [2] X. Zhu, X. Ao, Z. Qin, Y. Chang, Y. Liu, Q. He, and J. Li, “Intelligent financial fraud detection practices in post-pandemic era,” *The Innovation*, vol. 2, no. 4, p. 100176, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666675821001016>
- [3] A. Ali, S. Abd Razak, S. H. Othman, T. A. E. Eisa, A. Al-Dhaqm, M. Nasser, T. Elhassan, H. Elshafie, and A. Saif, “Financial fraud detection based on machine learning: A systematic literature review,” *Applied Sciences*, vol. 12, no. 19, p. 9637, 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/19/9637>
- [4] “Credit card fraud detection,” 2018. [Online]. Available: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [5] B. Krawczyk, “Learning from imbalanced data: open challenges and future directions,” *Progress in Artificial Intelligence*, vol. 5, pp. 221–232, 2016.
- [6] S. Rezvani and X. Wang, “A broad review on class imbalance learning techniques,” *Applied Soft Computing*, vol. 143, p. 110415, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494623004337>
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *J. Artif. Int. Res.*, vol. 16, no. 1, p. 321–357, 2002.
- [8] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, ser. NIPS’14. Cambridge, MA, USA: MIT Press, 2014, p. 2672–2680.
- [9] R. Sauber-Cole and T. M. Khoshgoftaar, “The use of generative adversarial networks to alleviate class imbalance in tabular data: a survey,” *J. Big Data*, vol. 9, no. 1, Aug. 2022.
- [10] J. Engelmann and S. Lessmann, “Conditional wasserstein gan-based oversampling of tabular data for imbalanced learning,” *Expert Systems with Applications*, vol. 174, p. 114582, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417421000233>

- [11] T. M. Khoshgoftaar, C. Seiffert, J. V. Hulse, A. Napolitano, and A. Folleco, "Learning with limited minority class data," in *Proceedings of the Sixth International Conference on Machine Learning and Applications*, ser. ICMLA '07. USA: IEEE Computer Society, 2007, p. 348–353. [Online]. Available: <https://doi.org/10.1109/ICMLA.2007.64>
- [12] U. Fiore, A. De Santis, F. Perla, P. Zanetti, and F. Palmieri, "Using generative adversarial networks for improving classification effectiveness in credit card fraud detection," *Information Sciences*, vol. 479, pp. 448–455, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025517311519>
- [13] F. AlMahamid and K. Grolinger, "Reinforcement learning algorithms: An overview and classification," in *2021 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*, 2021, pp. 1–7.
- [14] G. E. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD explorations newsletter*, vol. 6, no. 1, pp. 20–29, 2004.
- [15] A. Fernández, S. García, F. Herrera, and N. V. Chawla, "SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary," *J. Artif. Int. Res.*, vol. 61, no. 1, p. 863–905, jan 2018.
- [16] H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-smote: A new over-sampling method in imbalanced data sets learning," in *Advances in Intelligent Computing*, D.-S. Huang, X.-P. Zhang, and G.-B. Huang, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 878–887.
- [17] H. He, Y. Bai, E. A. Garcia, and S. Li, "Adasyn: Adaptive synthetic sampling approach for imbalanced learning," in *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, 2008, pp. 1322–1328.
- [18] C. Bunkhumpornpat, K. Sinapiromsaran, and C. Lursinsap, "Safe-Level-SMOTE: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem," in *Advances in Knowledge Discovery and Data Mining*, T. Theeramunkong, B. Kijssirikul, N. Cercone, and T.-B. Ho, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 475–482.
- [19] G. Mariani, F. Scheidegger, R. Istrate, C. Bekas, and C. Malossi, "Bagan: Data augmentation with balancing gan," 2018.
- [20] G. Douzas and F. Bacao, "Effective data generation for imbalanced learning using conditional generative adversarial networks," *Expert Systems with Applications*, vol. 91, pp. 464–471, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417417306346>
- [21] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in Neural Information Processing Systems*, 2019.

- [22] J. Wang and L. Yao, “Unrolled gan-based oversampling of credit card dataset for fraud detection,” in *2022 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA)*, 2022, pp. 858–861.
- [23] E. Strelcenia and S. Prakoonwit, “A new gan-based data augmentation method for handling class imbalance in credit card fraud detection,” in *2023 10th International Conference on Signal Processing and Integrated Networks (SPIN)*, 2023, pp. 627–634.
- [24] A. Sharma, P. K. Singh, and R. Chandra, “SMOTified-GAN for class imbalanced pattern classification problems,” *IEEE Access*, vol. 10, pp. 30 655–30 665, 2022.
- [25] F. A. Ghaleb, F. Saeed, M. Al-Sarem, S. N. Qasem, and T. Al-Hadhrani, “Ensemble synthesized minority oversampling-based generative adversarial networks and random forest algorithm for credit card fraud detection,” *IEEE Access*, vol. 11, pp. 89 694–89 710, 2023.
- [26] H. Ye, L. Xiang, and Y. Gan, “Detecting financial statement fraud using random forest with smote,” in *IOP Conference Series: Materials Science and Engineering*, vol. 612, no. 5. IOP Publishing, 2019, p. 052051.
- [27] C. Charitou, S. Dragicevic, and A. d. Garcez, “Synthetic data generation for fraud detection using gans,” *arXiv preprint arXiv:2109.12546*, 2021.
- [28] A. K. Shakya, G. Pillai, and S. Chakrabarty, “Reinforcement learning algorithms: A brief survey,” *Expert Systems with Applications*, vol. 231, p. 120495, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417423009971>
- [29] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [30] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Mach. Learn.*, vol. 8, no. 3-4, pp. 279–292, May 1992.
- [31] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing atari with deep reinforcement learning,” 2013.
- [32] D. S. Hado van Hasselt, Arthur Guez, “Deep reinforcement learning with double Q-learning,” vol. 30, 2016.
- [33] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Dueling network architectures for deep reinforcement learning,” 2016.
- [34] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Proceedings of the 12th International Conference on Neural Information Processing Systems*, ser. NIPS’99. Cambridge, MA, USA: MIT Press, 1999, p. 1057–1063.
- [35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017.
- [36] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” 2019.

- [37] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. P. Lillicrap, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48*, ser. ICML’16. JMLR.org, 2016, p. 1928–1937.
- [38] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” 2018.
- [39] M. Peng, Q. Zhang, X. Xing, T. Gui, X. Huang, Y.-G. Jiang, K. Ding, and Z. Chen, “Trainable undersampling for class-imbalance learning,” *Proc. Conf. AAAI Artif. Intell.*, vol. 33, no. 01, pp. 4707–4714, Jul. 2019.
- [40] J. Chi, G. Zeng, Q. Zhong, T. Liang, J. Feng, X. Ao, and J. Tang, “Learning to undersampling for class imbalanced credit risk forecasting,” in *2020 IEEE International Conference on Data Mining (ICDM)*, 2020, pp. 72–81.
- [41] S. Fan, X. Zhang, and Z. Song, “Imbalanced sample selection with deep reinforcement learning for fault diagnosis,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 4, pp. 2518–2527, 2022.
- [42] Y. Zhou, J. Shu, X. Zhong, X. Huang, C. Luo, and J. Ai, “Oversampling algorithm based on reinforcement learning in imbalanced problems,” in *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*, 2020, pp. 01–06.
- [43] D. Zha, K.-H. Lai, Q. Tan, S. Ding, N. Zou, and X. Hu, “Towards automated imbalanced learning with deep hierarchical reinforcement learning,” in *CIKM*, 2022.
- [44] E. Lin, Q. Chen, and X. Qi, “Deep reinforcement learning for imbalanced classification,” *Applied Intelligence*, vol. 50, no. 8, p. 2488–2502, 2020. [Online]. Available: <https://doi.org/10.1007/s10489-020-01637-z>
- [45] L. Zhinin-Vera, O. Chang, R. Valencia-Ramos, R. Velastegui, G. Pilliza, and F. So-casi, “Q-credit card fraud detector for imbalanced classification using reinforcement learning,” 02 2020, pp. 279–286.
- [46] S. Vimal, K. Kayathwal, H. Wadhwa, and G. Dhama, “Application of deep reinforcement learning to payment fraud,” 2021.
- [47] M. Jacob, G. S. H. Reddy, C. Rappai, P. Kapoor, and M. Kolhekar, “A comparative study of supervised and reinforcement learning techniques for the application of credit defaulters,” in *2022 IEEE 3rd Global Conference for Advancement in Technology (GCAT)*, 2022, pp. 1–6.
- [48] Q. Cui, L. Zhu, H. Feng, S. He, and J. Chen, “Intelligent fault quantitative identification via the improved deep deterministic policy gradient (ddpg) algorithm accompanied with imbalanced sample,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, pp. 1–13, 2023.
- [49] Y.-J. Su, L.-C. Wu, C.-H. Chen, and T.-Y. Chen, “Combining data resampling and drl algorithm for intrusion detection,” in *2023 5th International Conference on Computer Communication and the Internet (ICCCI)*, 2023, pp. 47–51.

- [50] X. Ma and W. Shi, “AESMOTE: Adversarial reinforcement learning with SMOTE for anomaly detection,” *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 2, pp. 943–956, 2021.
- [51] M. Qian and Y.-F. Li, “A novel adaptive undersampling framework for class-imbalance fault detection,” *IEEE Transactions on Reliability*, vol. 72, no. 3, pp. 1003–1017, 2023.
- [52] P. C. Y. Cheah, Y. Yang, and B. G. Lee, “Enhancing financial fraud detection through addressing class imbalance using hybrid SMOTE-GAN techniques,” *Int. J. Fin. Stud.*, vol. 11, no. 3, p. 110, Sep. 2023.
- [53] H. S. Jomaa, J. Grabocka, and L. Schmidt-Thieme, “Hyp-rl : Hyperparameter optimization by reinforcement learning,” 2019.
- [54] “South german credit,” 2019. [Online]. Available: <https://doi.org/10.24432/C5X89F>
- [55] C. Reinhart, K. Rogoff, C. Trebesch, and V. Reinhart, “Africa economic, banking and systemic crisis data,” 2019. [Online]. Available: <https://www.kaggle.com/datasets/chirin/africa-economic-banking-and-systemic-crisis-data>
- [56] “Fake bills,” 2019. [Online]. Available: <https://www.kaggle.com/datasets/alexandrepetit881234/fake-bills>
- [57] I.-C. Yeh, “Default of credit card clients,” 2016. [Online]. Available: <https://doi.org/10.24432/C55S3H>
- [58] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 507–520. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/0378c7692da36807bdec87ab043cdadc-Paper-Datasets_and_Benchmarks.pdf
- [59] R. Schwartz-Ziv and A. Armon, “Tabular data: Deep learning is not all you need,” *Inf. Fusion*, vol. 81, pp. 84–90, May 2022.
- [60] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
- [61] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>

Appendix A

Proposed RL Method's Performance

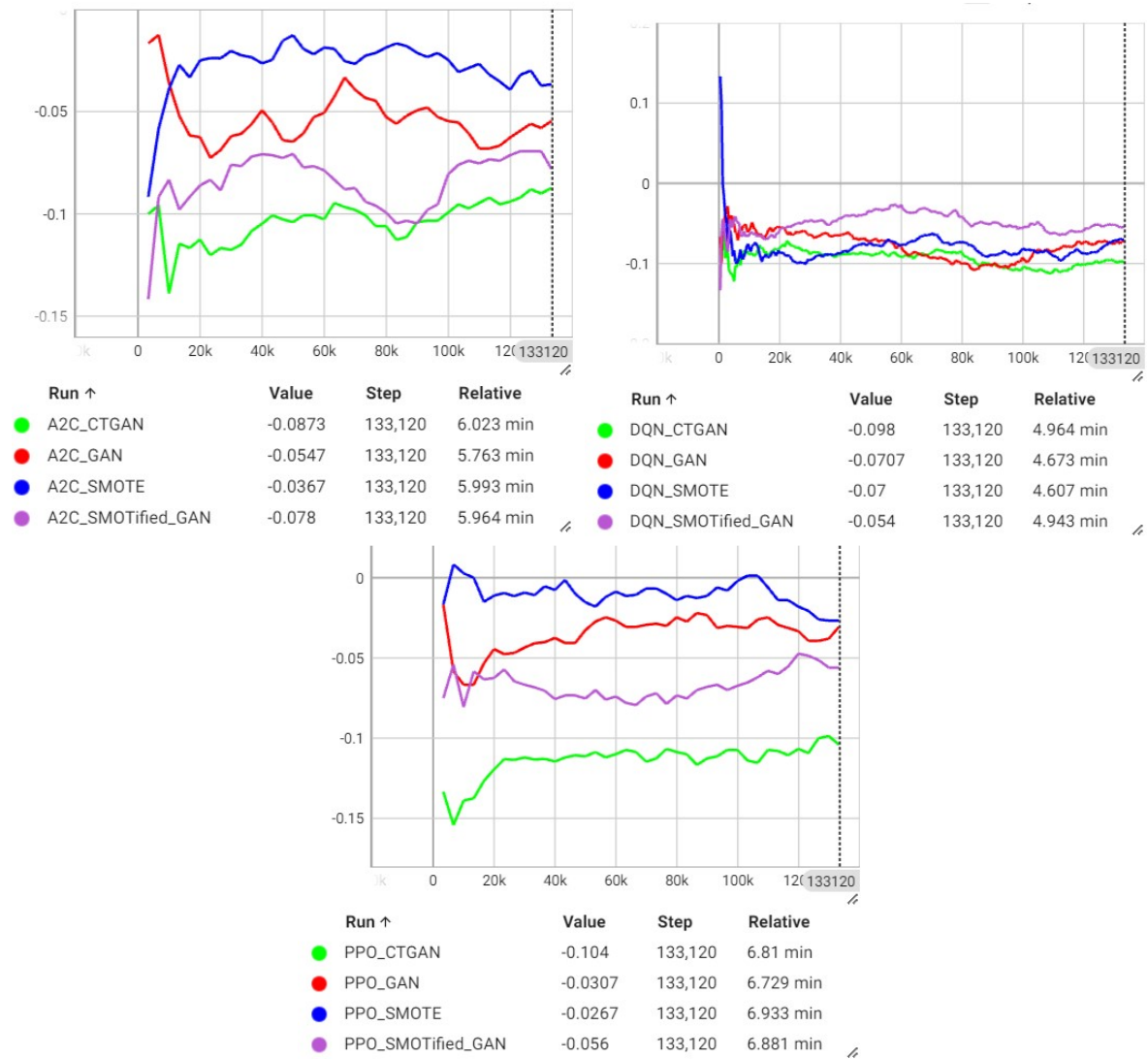


Figure A.1: Training Mean Episode Reward on South German Credit Dataset

Table A.1: Improvements (%) of The RL Sample Selection Method On Different M Values. Minor Improvement or Deterioration Between -1.0% and 1.0% (Exclusive) Were Omitted.

Dataset	Oversampling	RL	M=1.5				M=2.0				M=2.5				M=3.0			
			PR	RC	F1	AUC	PR	RC	F1	AUC	PR	RC	F1	AUC	PR	RC	F1	AUC
South German	SMOTE	A2C	-	-	-	-	-	-	-	-	-	-	-	-	1.7	-	-	-
		DQN	-1.0	-	-	-	-	1.0	-	-	-	-	-	-	-	-1.7	-	-1.1
		PPO	-	2.7	1.3	-	-	-	-	-	-	1.3	-	1.5	-	-4.0	-1.7	-
	GAN	A2C	-	-1.0	-	-	2.2	2.0	2.1	-	-	1.7	-	-	2.7	3.3	3.0	-
		DQN	-1.7	-	-	-1.2	-	-	-	-	-	1.7	-	-	3.7	2.0	2.9	-
		PPO	-1.3	-1.3	-1.3	-1.4	-	-	-	-1.3	-	2.0	1.1	-	2.4	1.7	2.1	-
	CTGAN	A2C	-2.4	-1.0	-1.8	-1.3	1.5	2.3	1.9	1.1	-2.2	-5.3	-3.6	-	-	2.7	1.2	-
		DQN	-1.4	-	-	-	-	-1.0	-	-	-1.0	-4.0	-2.4	-	-	-	-	-
		PPO	-2.6	-2.0	-2.3	-	-	2.0	-	1.4	-1.4	-5.0	-3.0	-	-	2.3	1.4	-
	SMOTified-GAN	A2C	-1.8	-	-	-	-	1.0	-	-	-	1.3	-	-	-2.8	-	-1.6	-
		DQN	-	-	-	-	2.4	5.7	3.9	-	2.6	2.0	2.3	-	-1.3	-1.7	-1.4	-
		PPO	-	1.0	-	1.1	2.7	4.3	3.5	-	1.9	-1.3	-	-	-3.3	-2.7	-3.1	-
African Crisis	SMOTE	A2C	-	-	-	-	9.6	-5.3	2.9	-	11.4	-5.3	3.8	-	9.9	-5.3	2.9	-
		DQN	10.1	-	5.6	-	4.6	-	2.7	-	5.5	-	3.2	-	5.6	-	3.2	-
		PPO	-	-	-	-	-	-5.3	-2.3	-	2.5	-4.2	-	-	-	-5.3	-2.2	-
	GAN	A2C	1.8	-	1.0	-	-	-1.1	-	-	-	-	-	-	-1.0	-1.1	-1.0	-
		DQN	1.8	-	1.0	-	-	-2.1	-1.1	-	-	-	-	-	-1.0	-1.1	-1.0	-
		PPO	-	-	-	-	-	-	-	-	-	1.1	-	-	-	1.1	-	-
	CTGAN	A2C	-	-3.2	-1.6	-	-	-1.1	-	-	-	-	-	-	-	-	-	-
		DQN	-3.5	1.1	-1.4	-	-	-	-	-	-	-	-	-	-	-	-	-
		PPO	-	2.1	1.1	-	-	-	-	-	-1.9	-1.1	-1.5	-	-	-	-	-
	SMOTified-GAN	A2C	-	-	-	-	-	1.1	-	-	-	-	-	-	-	-	-	-
		DQN	-	-	-	-	-	2.1	1.1	-	-1.0	-1.1	-1.0	-	-1.0	-1.1	-1.0	-
		PPO	-	-	-	-	-	1.1	-	-	-	-	-	-	-	-	-	-
Fake Bills	SMOTE	A2C	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		DQN	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		PPO	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	GAN	A2C	-1.1	-	-	-	-1.3	-	-	-	-	-	-	-	-	-	-	-
		DQN	2.0	-	-	-	3.8	-1.2	1.3	-	1.4	-	-	-	1.0	-	-	-
		PPO	2.4	-	1.3	-	3.8	-	1.5	-	1.4	-	-	-	1.0	-	-	-
	CTGAN	A2C	-	-	-	-	1.0	-	-	-	-	-	-	-	-1.5	-	-	-
		DQN	-	-	-	-	-	-	-	-	-1.6	-	-1.0	-	-2.1	-	-	-
		PPO	-	-	-	-	-1.2	-	-	-	-	-1.0	-	-	-	-	-	-
	SMOTified-GAN	A2C	2.1	-	-	-	1.4	-	-	-	-	-	-	-	-	-	-	-
		DQN	1.9	-	-	-	3.0	-	1.4	-	-	-	-	-	-	-	-	-
		PPO	-	-	-	-	-	1.0	-	-	-	-	-	-	-	1.0	-	-

Appendix B

Published Paper on IJFS



Article

Enhancing Financial Fraud Detection through Addressing Class Imbalance Using Hybrid SMOTE-GAN Techniques

Patience Chew Yee Cheah, Yue Yang and Boon Giin Lee *

School of Computer Science, University of Nottingham Ningbo China, Ningbo 315100, China; scypc3@nottingham.edu.cn (P.C.Y.C.); yue.yang2@nottingham.edu.cn (Y.Y.)

* Correspondence: boon-giin.lee@nottingham.edu.cn

Abstract: The class imbalance problem in finance fraud datasets often leads to biased prediction towards the nonfraud class, resulting in poor performance in the fraud class. This study explores the effects of utilizing the Synthetic Minority Oversampling TEchnique (SMOTE), a Generative Adversarial Network (GAN), and their combinations to address the class imbalance issue. Their effectiveness was evaluated using a Feed-forward Neural Network (FNN), Convolutional Neural Network (CNN), and their hybrid (FNN+CNN). This study found that regardless of the data generation techniques applied, the classifier's hyperparameters can affect classification performance. The comparisons of various data generation techniques demonstrated the effectiveness of the hybrid SMOTE and GAN, including SMOTified-GAN, SMOTE+GAN, and GANified-SMOTE, compared with SMOTE and GAN. The SMOTified-GAN and the proposed GANified-SMOTE were able to perform equally well across different amounts of generated fraud samples.

Keywords: class imbalance; data generation; deep learning; financial fraud detection



Citation: Cheah, Patience Chew Yee, Yue Yang, and Boon Giin Lee. 2023. Enhancing Financial Fraud Detection through Addressing Class Imbalance Using Hybrid SMOTE-GAN Techniques. *International Journal of Financial Studies* 11: 110. <https://doi.org/10.3390/ijfs11030110>

Academic Editors: Albert Y.S. Lam and Yanhui Geng

Received: 27 July 2023

Revised: 29 August 2023

Accepted: 1 September 2023

Published: 5 September 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The financial sector faces a significant challenge in the form of financial fraud, encompassing various forms of criminal deception aimed at securing financial gains, including activities like telecommunication fraud and credit card skimming. The proliferation of electronic payment technology has propelled online transactions into the mainstream, thereby amplifying the occurrence of fraudulent schemes. The prevalence of these fraudulent transactions has led to substantial losses for financial institutions. However, the large daily transactions pose a challenge for humans in manually identifying fraud. Recently, deep learning techniques have been explored and have shown promising results in detecting financial fraud [Alarfaj et al. \(2022\)](#); [Fang et al. \(2021\)](#); [Kim et al. \(2019\)](#). Unfortunately, most real-world financial fraud datasets suffer from a severe class imbalance issue, where the fraud data's proportion is significantly lower than that of nonfraud. In binary classification, class imbalance often leads to biased predictions favoring the majority class [Johnson and Khoshgoftaar \(2019\)](#). Consequently, the classifier's performance on the minority class is compromised, especially when encountering dissimilar frauds. Overcoming this problem poses a significant challenge, as classifiers are expected to achieve high precision and recall in fraudulent class.

To address this problem, several oversampling methods have been employed to generate minority samples. Synthetic Minority Oversampling TEchnique (SMOTE) interpolates between the existing minority data to synthesize minority samples [Chawla et al. \(2002\)](#). Generative Adversarial Networks (GANs) comprise a discriminator that aims to differentiate between real and generated samples and a generator that strives to deceive the discriminator by synthesizing realistic samples [Goodfellow et al. \(2014\)](#). GANs have shown superior results compared with SMOTE [Fiore et al. \(2019\)](#). However, SMOTE may cause overgeneralization issues. GAN, primarily designed for image generation, is not ideal

for handling the class imbalance problem. To overcome these limitations, SMOTified-GAN employs SMOTE-generated samples instead of random noises as input to the GAN [Sharma et al. \(2022\)](#).

In addition to the aforementioned data generation techniques, other hybrids of SMOTE and GAN are worth exploring. This study presents the following contributions:

1. Introducing two data generation techniques, SMOTE+GAN and GANified-SMOTE, designed to effectively address the class imbalance issue in finance fraud detection.
2. Conducting a comprehensive comparison between the proposed oversampling methods and existing data generation techniques, utilizing precision, recall, and F1-score as key performance metrics.
3. Evaluating the performance of the data generation techniques across various neural network architectures, including a Feed-forward Neural Network (FNN), Convolutional Neural Network (CNN), and the proposed hybrid FNN+CNN.
4. Analyzing the impact of training classifiers on different proportions of the generated minority samples.

2. Related Work

The task of detecting financial fraud can be approached as a binary classification challenge, where classifiers examine the patterns within fraudulent and legitimate transactions to classify new transactions accurately. Consequently, it is crucial to possess an ample and diverse dataset to enable classifiers to grasp the inherent patterns of both transaction categories. Addressing the issue of inadequate fraudulent samples in the training dataset, various methodologies have been introduced to create artificial fraud instances and supplement the original data. These techniques include SMOTE, GAN, and SMOTified-GAN.

SMOTE [Chawla et al. \(2002\)](#) has been widely applied to imbalanced training datasets. More than 85 SMOTE variations were proposed by 2018, including SMOTE+TomekLinks, SMOTE+ENN, Borderline-SMOTE, and Adaptive Synthetic [Fernández et al. \(2018\)](#). Recent studies proposed Radius-SMOTE [Pradipta et al. \(2021\)](#), which prevents overlap among generated samples, and Reduced-Noise SMOTE [Arafa et al. \(2022\)](#), which removes noise after oversampling. In financial fraud detection, SMOTE and its variations have been widely utilized to resample highly imbalanced datasets before training models such as AdaBoost [Ileberi et al. \(2021\)](#) and FNN [Fang et al. \(2021\)](#). Besides the finance domain, SMOTE and its variations have found extensive application in other fields dealing with highly imbalanced datasets. In bio-informatics, SMOTE has been used to discriminate Golgi proteins [Tahir et al. \(2020\)](#) and predict binding hot spots in protein–RNA interactions [Zhou et al. \(2022\)](#). In medical diagnosis, SMOTE and its variations have been employed for diagnosing cervical cancer [Abdoh et al. \(2018\)](#) and prostate cancer [Abraham and Nair \(2018\)](#). SMOTE has also been used to predict diabetes [Mirza et al. \(2018\)](#) and heart failure patients' survival [Ishaq et al. \(2021\)](#).

GANs [Goodfellow et al. \(2014\)](#) and their variations have more recently been employed for generating minority samples to tackle the class imbalance problem. [Douzas and Bacao \(2018\)](#) utilized a conditional GAN (cGAN) which can recover the distribution of training data to generate minority samples. To address the mode collapse issue, Balancing GAN was proposed to generate more diverse and higher-quality minority images [Mariani et al. \(2018\)](#). However, in this technique, the generator and discriminator cannot simultaneously reach their optimal states, leading to the development of IDA-GAN [Yang and Zhou \(2021\)](#). In financial fraud detection, GAN has been employed to generate fraud samples for imbalanced datasets before training classifiers, such as AdaBoost-Decision Tree [Mo et al. \(2019\)](#) and FNN [Fiore et al. \(2019\)](#). These studies have reported that the GAN achieves higher AUC, accuracy, and precision compared with SMOTE. Interestingly, [Fiore et al. \(2019\)](#) found that the best performance was achieved when twice as many GAN-generated fraud samples as the original fraud data were added to the training dataset. In other finance-related domains, GANs have been utilized to address class imbalance in money laundering detection in gambling [Charitou et al. \(2021\)](#). GANs and their variations have also been used

extensively for high-dimensional imbalanced datasets, such as images [Mariani et al. \(2018\)](#); [Scott and Plested \(2019\)](#) and biomedical data [Zhang et al. \(2018\)](#). Recent studies have successfully applied GANs and their variations to generate minority samples in bio-informatics [Lan et al. \(2020\)](#).

Despite the notable accomplishments of SMOTE and GAN, these methods have certain limitations. SMOTE may introduce noise that leads to overgeneralization [Bunghumpornpat et al. \(2009\)](#). While GANs can generate more “realistic” data, they may not be ideal for handling imbalanced data, as it was originally designed for generating images using random noise. Additionally, there may be insufficient real minority data available for training the GAN [Mariani et al. \(2018\)](#). To address these limitations, [Sharma et al. \(2022\)](#) proposed SMOTified-GAN, which employs SMOTE-generated samples as input for GAN instead of random numbers, resulting in improved performance compared with SMOTE and GAN.

In early studies, financial fraud detection systems predominantly depended on rule-based methodologies, wherein human expertise in fraud was translated into rules to anticipate fraudulent activities [Zhu et al. \(2021\)](#). However, the evolving behaviors of fraudsters and the increasing size of transaction datasets have posed challenges in identifying fraud-related rules manually. As a result, research has shifted towards machine learning methods, such as naive Bayes, logistic regression, support vector machine, random forest, and decision tree ([Ileberi et al. 2021](#); [Ye et al. 2019](#); [Zhu et al. 2021](#)), which can “learn” fraud and nonfraud patterns from given datasets. Nonetheless, machine learning techniques require extensive data preprocessing before training the classifier [Alarfaj et al. \(2022\)](#); [Kim et al. \(2019\)](#); [Zhu et al. \(2021\)](#).

In recent years, deep learning has gained popularity in financial fraud detection due to its superior performance compared with traditional machine learning approaches [Alarfaj et al. \(2022\)](#); [Fang et al. \(2021\)](#); [Jurgovsky et al. \(2018\)](#); [Kim et al. \(2019\)](#). Some studies have approached financial fraud detection as a sequence classification problem, considering the temporal sequence of transactions as a crucial factor. Sequential models, such as Gated Recurrent Units [Branco et al. \(2020\)](#), Long Short-Term Memory (LSTM) [Jurgovsky et al. \(2018\)](#), and Time-aware Attention-based Interactive LSTM [Xie et al. \(2022\)](#), have been proposed. However, since most available financial fraud datasets lack time-sequence information, sequential models may not be suitable in such cases. Due to the vector format of finance fraud datasets without time-sequence information, FNNs are considered a suitable choice [Fang et al. \(2021\)](#); [Fiore et al. \(2019\)](#); [Kim et al. \(2019\)](#). Initially designed for image processing and classification, CNNs have also been found effective in financial fraud detection [Alarfaj et al. \(2022\)](#); [Chen and Lai \(2021\)](#); [Zhang et al. \(2018\)](#). Their 1D convolution layers can extract patterns within smaller segments of a transaction vector.

Building on [Fiore et al. \(2019\)](#)’s findings, this study aimed to assess the performance of a model using varying amounts of minority samples in the training dataset. To achieve this, the study explores the use of SMOTE, GAN, SMOTified-GAN, and other variants of hybrid SMOTE and GAN. Consequently, a combination of SMOTE- and GAN-generated minority samples, along with GANified-SMOTE, was proposed to fulfill the research aims. Finally, FNN, CNN, and FNN+CNN models were employed to ensure a fair evaluation of the performances of different data generation techniques.

3. Methodology

3.1. Data Preprocessing

The experiment utilized the [Kaggle \(2018\)](#) credit card fraud dataset, consisting of 284,807 transactions conducted by European credit card holders over two days in September 2013. This dataset comprises 31 numerical features, including Time, Amount, Class, and 28 other unnamed features. The ‘Time’ feature represents the elapsed time in seconds since the first transaction, while the ‘Amount’ feature denotes the transaction amount. The ‘Class’ label indicates fraudulence, utilizing binary values, where 1 and 0 represent fraud and

nonfraud, respectively. Notably, only 492 transactions (0.172%) are classified as fraudulent, resulting in a highly imbalanced distribution.

To facilitate gradient descent convergence and mitigate bias towards features with larger magnitudes, all features except the 'Class' label were rescaled to the range $[0, 1]$ while maintaining the original feature distribution. This rescaling process for a value X in a given feature was transformed into a new value X' (see Equation (1), where X_{min} and X_{max} represent the minimum and maximum values of the feature, respectively) to maintain the original feature distribution.

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (1)$$

Subsequently, the dataset was divided into a training set comprising 80% of the data (227,451 nonfraud and 394 fraud) and a testing set comprising the remaining 20% (56,864 nonfraud and 98 fraud).

3.2. Data Generation Methods

To address the issue of class imbalance, this study explored five data generation techniques: SMOTE, GAN, and their respective combinations.

3.2.1. SMOTE

SMOTE creates synthetic minority samples rather than duplicating existing ones to avoid overfitting. For a specific minority data point x represented as a vector, a vector x_k is randomly chosen from its k -nearest neighbors to generate a new sample x' using Equation (2). In this study, 394 instances of fraudulent data from the training dataset were utilized with the SMOTE technique, employing five nearest neighbors, to generate additional fraud samples, as depicted in Figure 1.

$$x' = x + rand(0, 1) \times (x - x_k) \quad (2)$$

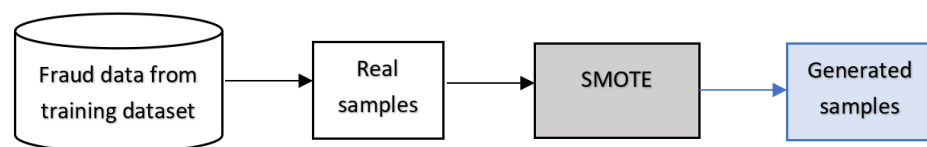


Figure 1. SMOTE employed in this study utilizing five nearest neighbors for random interpolations and generating minority samples.

3.2.2. GAN

A GAN comprises a generator G and a discriminator D that engage in a competitive training process to improve their respective objectives. The discriminator aims to correctly classify real samples x and fake samples generated by the $G(z)$, where z represents random noise or the latent space input to the G . The D 's predictions for real and generated samples are denoted as $D(x)$ and $D(G(z))$, respectively. By considering real samples with a label of 1 and generated samples with a label of 0, the D 's loss function is defined in Equation (3), where E calculates the error or distance between the D 's prediction and the true label. The G 's objective is to generate realistic fake samples from random noise that can deceive the D into misclassifying them. The G 's general loss function, defined in Equation (4), allows it to improve the quality of the generated samples based on the feedback received from the D 's classification. As the G and D continue enhancing their performance, the quality of the generated minority samples improves.

$$L_D = E(D(x), 1) + E(D(G(z)), 0) \quad (3)$$

$$L_G = E(D(G(z)), 1) \quad (4)$$

The proposed GAN architecture, as shown in Figure 2, consists of a 5-layer FNN G with respective neuron counts of 100, 256, 128, 64, and 30. The G takes 100 random noises sampled from a normal distribution. LeakyReLU activation function (Equation (5)) is used in all hidden layers, and dropout layers with a dropout rate of 0.2 are added after each hidden layer to mitigate overfitting. The output layer employs the sigmoid activation function (Equation (6)) to produce values between 0 and 1. Similarly, the D is a 5-layer FNN with identical activation functions and dropout layers. However, the neuron counts are 30, 128, 64, 32, and 1 for each layer. The D employs a stochastic gradient descent (SGD) optimizer with a learning rate of 0.05. The loss function depicted in Figure 2 is binary cross-entropy (Equation (7)), as the D 's task involves binary classification. The GAN network also employs an optimizer with the same learning rate as the D , but the loss function utilizes the mean squared error metric (Equation (8), where y_i is the true label and $\hat{y}_i$ is the predicted class) as feedback for the G .

$$f(x) = \max(0.1x, x) \quad (5)$$

$$S(x) = \frac{1}{1 + e^{-x}} \quad (6)$$

$$\text{BinaryCrossEntropy} = \frac{1}{n} \sum_{i=1}^n [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)] \quad (7)$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (8)$$

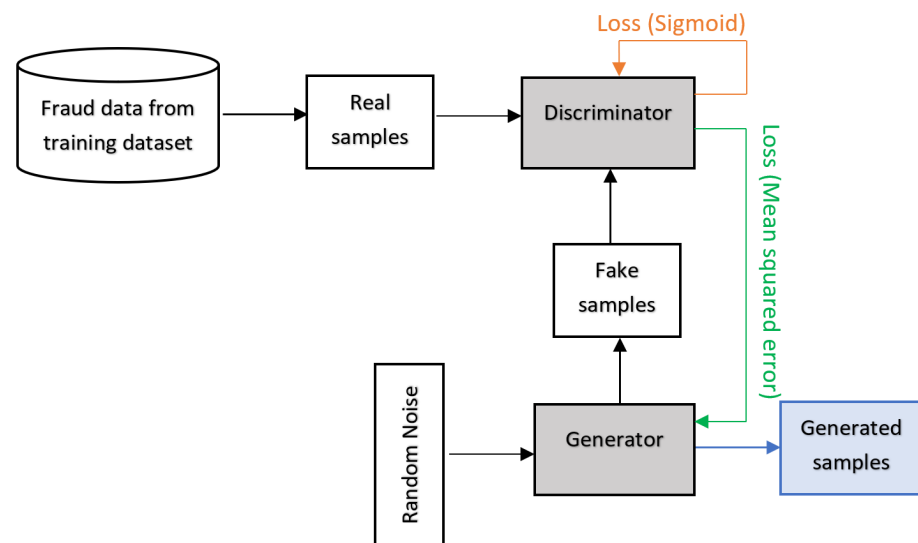


Figure 2. GAN architecture employed in this study consisting of a 5-layer FNN generator and discriminator, leading to the generation of the final minority samples depicted by the blue square.

The fraud data from the training dataset were utilized to train D , enabling it to recognize patterns in real fraud data and generate fraud samples. Since there were only 394 fraudulent data points available for training, the batch size was reduced to 32. The number of training epochs was set to 1000 to allow sufficient time for the G and D to improve their performance. Following training, the G is employed to generate fraud samples based on the required number of minority samples.

3.2.3. SMOTified-GAN

GAN can learn patterns from minority data, resulting in more authentic minority samples. However, using random noise as input for the GAN G can be seen as generating samples from scratch, making it more challenging to train the G to produce high-quality

samples. By utilizing SMOTE-generated samples as input, the generation process becomes simpler as the G begins with pre-existing fraud samples (Sharma et al. 2022). In the proposed approach, SMOTE was applied with the five nearest neighbors to generate double the number of fraud samples. Figure 3 illustrates that 788 SMOTE-generated samples were used as input for the GAN G . The hyperparameters of the GAN in the SMOTified-GAN model remained the same as the regular GAN, except for the number of neurons in the input layer of the G , which was adjusted to 30 to match the 30 features present in the SMOTE-generated fraud samples.

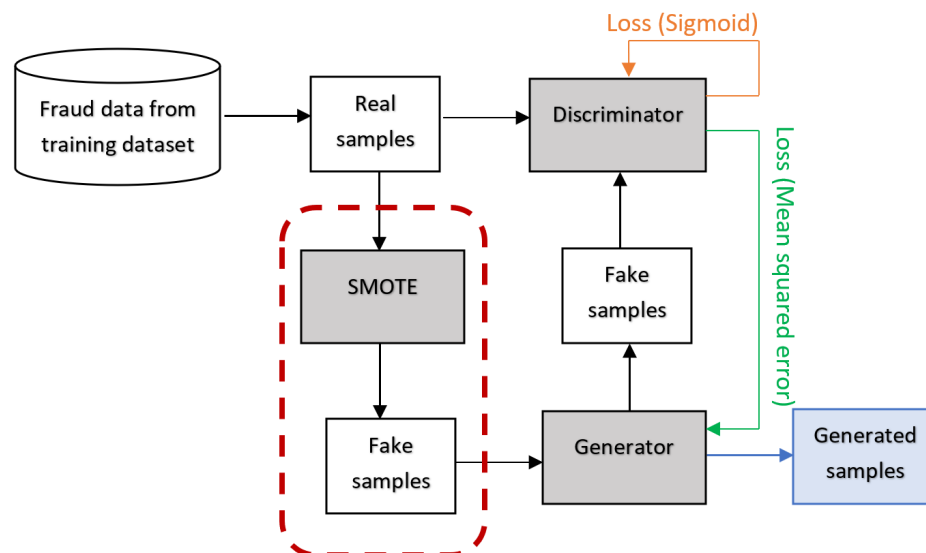


Figure 3. SMOTified-GAN architecture that employed SMOTE-generated samples as the input for the G , deviating from the traditional GAN approach that uses random noise. The final minority samples were produced and depicted in the blue square.

3.2.4. SMOTE+GAN

To address the limitations of SMOTE and GAN, a hybrid approach was proposed and employed to enhance the ratio of fraudulent data in the training dataset. The SMOTE-generated and GAN-generated fraud samples were directly combined with the original training dataset without any alterations, as depicted in Figure 4. The combined dataset comprised an equal contribution from both the SMOTE- and GAN-generated samples, amounting to half of the total required generated data.

3.2.5. GANified-SMOTE

Another hybrid method, GANified-SMOTE, was implemented. The random interpolation makes SMOTE-generated samples susceptible to the noise present in the dataset. Consequently, the generated minority samples are located near the boundary of the majority class, leading to higher misclassification rates. Conversely, GAN can learn the underlying patterns of the minority class, reducing the impact of such noise. By utilizing GAN-generated data for SMOTE interpolations, the limitations of SMOTE can be overcome. Additionally, applying SMOTE on the GAN-generated data can decrease reliance on the prominent patterns of the minority class, thereby mitigating overfitting. Figure 5 illustrates the utilization of fraud samples generated by the GAN, which are then processed with SMOTE to generate the necessary number of fraud samples. The resulting output from SMOTE is combined with the original training dataset, which originally contained 394 authentic fraud data points.

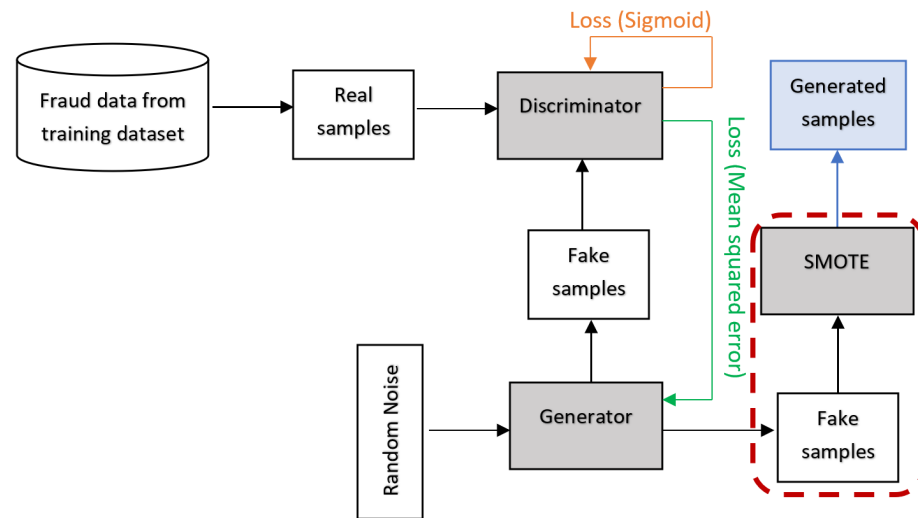


Figure 4. SMOTE+GAN architecture was employed to generate minority samples by directly incorporating SMOTE and GAN techniques. These generated samples were then merged with the original training dataset.

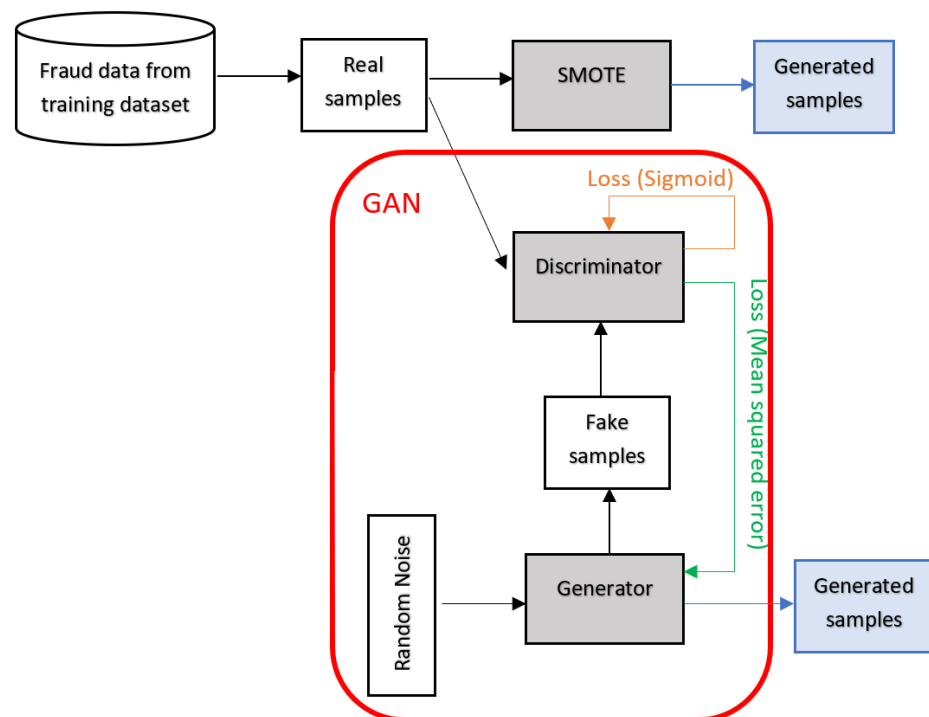


Figure 5. Proposed GANified-SMOTE architecture that involved the application of SMOTE to the GAN-generated samples, as indicated by the red dashed square.

3.3. Summary of Data Generation Methods

Table 1 presents an overview of the types and quantities of fraud data utilized in each data generation method. Two experiments were conducted for each method to assess the impact of varying amounts of generated data in the training dataset. In the first experiment (Test A), the training dataset was adjusted to achieve a balanced distribution of 50% fraud and 50% nonfraud samples. In the second experiment (Test B), only 788 fraud samples were generated, twice the number of the original fraud data in the training dataset. This choice was based on the finding (Fiore et al. 2019) that injecting twice as many GAN-generated fraud samples as the original fraud data produced the optimal outcome.

For both experiments, fraud samples were generated using five data generation techniques after splitting the complete dataset into training and testing sets. The testing dataset was not utilized for data generation to ensure that the validation conducted using these unseen data reflects the model's performance when applied to real-world financial fraud detection systems, as these systems encounter unseen data.

Table 1. Variations in the types and quantities of fraud data utilized for each data generation method.

Generation Method	Types of Fraud Data	Total Fraud Data	Total Generated Fraud Data	
			Test A	Test B
SMOTE	Real	394	227,057	788
GAN	Real	394	227,057	788
SMOTified-GAN	Real SMOTE	394 788	227,057	788
SMOTE+GAN	Real	394	SMOTE: 113,529 GAN: 113,529	SMOTE: 344 GAN: 344
GANified-SMOTE	Real SMOTE	394 394	227,057	788

3.4. Deep Learning Models

3.4.1. FNN

A preliminary investigation was conducted to assess several hyperparameter configurations of the FNN (Feed-forward Neural Network) along with SMOTE-generated samples to tackle the problem of class imbalance. The two most effective models were selected as classifiers to evaluate all the data generation techniques. Table 2 contains the hyperparameters used for these models. Both models employed the Rectified Linear Unit (ReLU) activation function for their hidden layers. To counter overfitting, dropout layers with a dropout rate of 0.1 were inserted after each hidden layer. The output layer utilized the sigmoid activation function to ensure that the output probabilities fall within the range of 0 to 1, representing the likelihood of a transaction being fraudulent. For the loss function, binary cross-entropy was employed. Due to the substantial size of the training dataset, a batch size of 128 was chosen, and the training process was executed over 100 epochs, allowing for multiple iterations to refine the model.

Table 2. The hyperparameters of the two top performing models within FNN and CNN variations.

Model	Layers	Neurons in Each Layer	Filters	Optimizer	Learning Rate
FNN1	6 DENSE	30-200-100-50-10-1	-	SGD	0.01
FNN2	6 DENSE	30-200-100-50-10-1	-	SGD	0.05
CNN1	1 CONV+1 POOL	-	64	Adam	0.01
CNN2	1 CONV+1 POOL	-	32	Adam	0.01

3.4.2. CNN

Similarly to the FNN, various hyperparameter configurations of the CNN were tested, and the two best-performing models were selected for further investigation. The hyperparameters for these models are presented in Table 2. Both models began with an input layer of dimensions (30, 1). Subsequently, a 1D convolutional layer and a max-pooling layer were incorporated, followed by a flattening layer and a dense layer consisting of 50 neurons, utilizing the ReLU activation function, along with a dropout layer featuring a dropout rate of 0.1. The output layer consisted of a single neuron activated by the sigmoid function. The kernel size and pool size for both models were set to 3 and 2, respectively. The initial findings indicated that the CNN models reached a stable loss and accuracy after the 50th

epoch. Consequently, the number of training epochs was set to 50, providing sufficient time to refine the models and observe their performance.

3.4.3. FNN+CNN

FNN and CNN models tend to misclassify nonfraudulent transactions, while demonstrating an intuitive ability to identify the same fraudulent transactions. Consequently, this study integrated the two models to enhance the final prediction, aiming to reduce the false-positive rate within the fraud class. By leveraging the strengths of both models and combining their insights, it was anticipated that the integrated approach would yield improved accuracy and more reliable identification of fraudulent transactions.

Method I: The final prediction is classified as fraudulent only if both FNN and CNN predict the transaction as fraudulent. The detailed processes and decision steps of this method are depicted in Figure 6a. Both the FNN and CNN output a probability of a transaction being fraudulent, where a value greater than 0.5 is considered fraudulent. Hence, the integrated model predicted a transaction as fraudulent only if both models' output surpassed 0.5. Intuitively, it is improbable for a nonfraudulent transaction to be classified as fraudulent by both models, given their tendency to learn distinct patterns associated with fraud and nonfraud. This integration reinforces the reliability of the fraud prediction, since it must satisfy both conditions.

Method II: The initial study demonstrated that the first method successfully enhanced the precision of fraud detection but resulted in a decrease in the recall. Therefore, another method was proposed to increase recall while maintaining a high precision. In certain instances, one of the models produces a value close to 1, indicating a high probability of the transaction being fraudulent. Conversely, the other model generates a value below but close to 0.5. According to the first method, these transactions would be predicted as nonfraudulent. However, intuitively, such transactions are more likely to be fraud, since one of the models strongly indicates fraud. To address this scenario, the sum of the output values from both models is utilized to make the final prediction. If the sum exceeds a selected threshold, the prediction will be fraudulent. The detailed processes and decision steps of this method are depicted in Figure 6b.

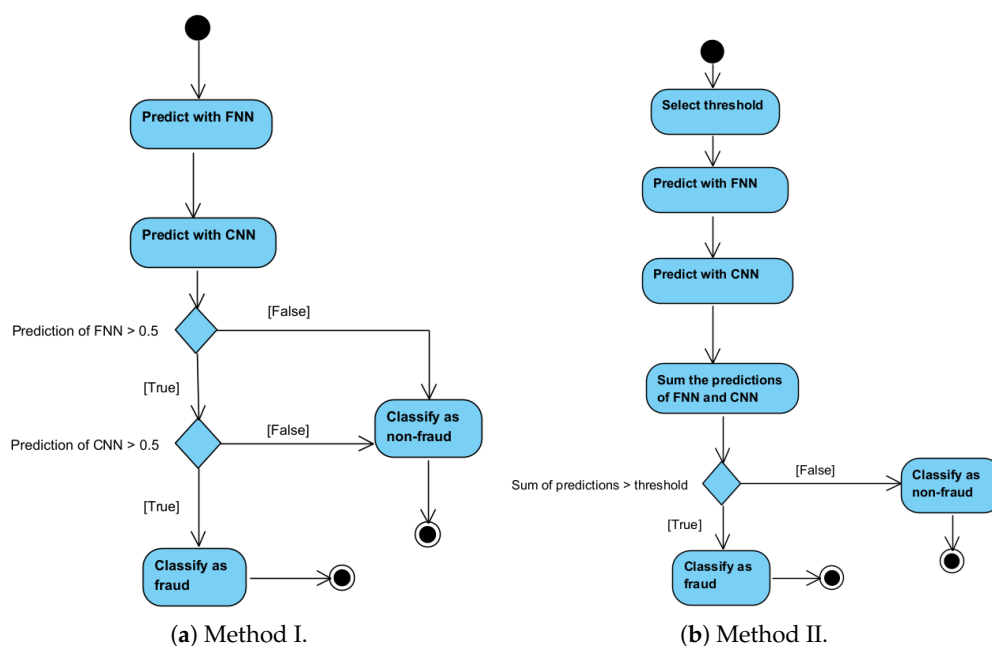


Figure 6. Flowchart illustrating the two methods employed for FNN+CNN.

4. Results and Discussion

4.1. Deep Learning Models with SMOTE-Generated Data

In this study, two FNN and two CNN models were developed to determine the optimal configuration, and their specific hyperparameters are outlined in Table 2. The training process took place on a machine equipped with an 11th Gen Intel Core i7-11375H CPU, 16GB RAM, Intel Iris Xe Graphics, and NVIDIA GeForce RTX 3060 Laptop GPU. The training dataset consisted of an equal distribution of fraud and nonfraud data, with the fraud samples generated using the SMOTE technique (refer to Section 3.2.1). To evaluate their performance, these variations were tested on the testing dataset, and various metrics were employed, including training accuracy, loss, and time, as well as testing precision (PR), recall (RC), F1-score (F1), and root mean squared error (RMSE).

In Table 3, all the top-performing models achieved impeccable precision, recall, and F1-score ($PR = RC = F1 = 1.00$) for the nonfraud class, and their recalls in the fraud class were satisfactory ($RC \geq 0.85$). However, their precision and F1-score in the fraud class did not meet the desired criteria. FNN1, which utilized a lower learning rate, demonstrated higher precision and F1-score compared with FNN2. Similarly, CNN2, with fewer filters, exhibited higher precision and F1-score compared with CNN1, albeit with a slightly lower precision. Consequently, FNN2 and CNN2, boasting the highest F1-score within their respective FNN and CNN models, were selected as the top-performing models for integration (FNN+CNN). It is worth noting that FNN models using the SGD optimizer yielded superior results compared with those employing Adam, while the opposite was observed for CNN. Additionally, CNN's training time was longer than that of FNN due to the fewer epochs utilized.

Table 3. The results of the two top-performing models within FNN and CNN variations are presented. The selection of the best model was based on comparing their recalls first, followed by their precision.

Model	TA ¹	TL ²	TT ³	PR ⁴		RC ⁵		F1 ⁶		RMSE ⁷
				NF ⁸	F ⁹	NF ⁸	F ⁹	NF ⁸	F ⁹	
FNN1	0.9952	0.0169	27 m 00 s	1.00	0.25	1.00	0.91	1.00	0.39	0.0692
FNN2	0.9961	0.0137	27 m 22 s	1.00	0.29	1.00	0.90	1.00	0.44	0.0624
CNN1	0.9960	0.0196	13 m 00 s	1.00	0.29	1.00	0.89	1.00	0.43	0.0634
CNN2	0.9981	0.0123	12 m 54 s	1.00	0.47	1.00	0.85	1.00	0.60	0.0437

¹ Training Accuracy, ² Training Loss, ³ Training Time, ⁴ Precision, ⁵ Recall, ⁶ F1-score, ⁷ Root Mean Square Error, ⁸ Nonfraud, ⁹ Fraud.

4.2. Deep Learning Models with Data Generation Methods

4.2.1. FNN and CNN

This research employed four selected FNNs and CNNs to evaluate the impact and performance of the generated data. The results can be found in Table 4. When employing the identical data generation method and incorporating an equal quantity of fraudulent data, both versions of FNN and CNN yielded comparable outcomes. This demonstrates that the outcomes derived from a data generation method are not significantly influenced by the classifier's parameters.

Table 4. The outcomes of employing five methods to generate data on four models were evaluated on two distinct minority samples.

Model	Generation Method	Test Samples	TA ¹	TL ²	PR ³	RC ⁴	F1 ⁵
FNN1	SMOTE	A	0.9952	0.0169	0.25	0.91	0.39
		B	0.9994	0.0028	0.81	0.88	0.84
	GAN	A	0.9995	0.0036	0.84	0.88	0.86
		B	0.9994	0.0025	0.81	0.88	0.84
	SMOTified-GAN	A	0.9995	0.0024	0.85	0.87	0.86
		B	0.9995	0.0024	0.83	0.86	0.84
	SMOTE+GAN	A	0.9966	0.0119	0.32	0.90	0.48
		B	0.9995	0.0025	0.83	0.88	0.85
	GANified-SMOTE	A	0.9994	0.0038	0.81	0.87	0.84
		B	0.9995	0.0025	0.82	0.88	0.85
FNN2	SMOTE	A	0.9961	0.0137	0.29	0.90	0.44
		B	0.9995	0.0026	0.85	0.88	0.86
	GAN	A	0.9995	0.0034	0.83	0.88	0.85
		B	0.9994	0.0027	0.81	0.88	0.84
	SMOTified-GAN	A	0.9996	0.0023	0.87	0.88	0.87
		B	0.9995	0.0024	0.87	0.87	0.87
	SMOTE+GAN	A	0.9956	0.0141	0.27	0.91	0.41
		B	0.9995	0.0025	0.85	0.88	0.86
	GANified-SMOTE	A	0.9996	0.0029	0.88	0.86	0.87
		B	0.9995	0.0022	0.87	0.87	0.87
CNN1	SMOTE	A	0.9960	0.0196	0.29	0.89	0.43
		B	0.9995	0.0029	0.85	0.87	0.86
	GAN	A	0.9995	0.0042	0.85	0.87	0.86
		B	0.9995	0.0031	0.86	0.86	0.86
	SMOTified-GAN	A	0.9995	0.0028	0.90	0.83	0.86
		B	0.9994	0.0032	0.80	0.84	0.82
	SMOTE+GAN	A	0.9962	0.0140	0.29	0.86	0.44
		B	0.9996	0.0025	0.87	0.88	0.87
	GANified-SMOTE	A	0.9995	0.0045	0.85	0.84	0.85
		B	0.9994	0.0034	0.79	0.86	0.82
CNN2	SMOTE	A	0.9981	0.0123	0.47	0.85	0.60
		B	0.9995	0.0034	0.83	0.86	0.84
	GAN	A	0.9994	0.0040	0.82	0.87	0.84
		B	0.9995	0.0029	0.86	0.84	0.85
	SMOTified-GAN	A	0.9995	0.0026	0.89	0.83	0.86
		B	0.9994	0.0036	0.81	0.88	0.84
	SMOTE+GAN	A	0.9965	0.0181	0.32	0.90	0.47
		B	0.9996	0.0039	0.88	0.86	0.87
	GANified-SMOTE	A	0.9995	0.0035	0.89	0.83	0.86
		B	0.9996	0.0023	0.91	0.83	0.87

¹ Training Accuracy, ² Training Loss, ³ Precision, ⁴ Recall, ⁵ F1-score. The top-performing PR, RC, and F1-score for both Test A and Test B are highlighted in bold, except for the RC in Test B, as identifying the best result posed a challenge due to high similarities.

The FNN generally yielded a higher recall compared with the CNN, albeit with lower precision. In test A, both SMOTE and SMOTE+GAN exhibited significantly lower precision and F1-score across all models, despite demonstrating a high recall. However, there was substantial improvement observed in test B for these two methods, where synthetic fraud samples were injected at a ratio of twice the original records. Additionally, SMOTE+GAN

achieved the highest F1-score in three out of four models during test B. The proposed GANified-SMOTE generally yielded slightly higher precision and F1-score than GAN, despite having a lower recall. This can be attributed to GAN's ability to capture the original fraud data's characteristics, resulting in the GAN-generated fraud samples being clustered in regions with a high concentration of the original fraud data. The SMOTE's application on the GAN-generated samples generates additional fraud samples in between them, potentially causing a 'blurring' effect on the fraud data's features. This could explain the generally lower recall of GANified-SMOTE in comparison with GAN. As a trade-off, GANified-SMOTE achieves a lower false positive rate, leading to higher precision compared with GAN. When compared with SMOTified-GAN, the proposed GANified-SMOTE generally demonstrated slightly lower precision and F1-score, while maintaining a similar recall. SMOTified-GAN generates fraud samples using SMOTE-generated samples as input, which can result in the production of more realistic and diverse samples. The SMOTified-GAN-generated samples tended to be more centrally distributed within fraud areas and less centrally distributed within nonfraud areas, which could explain the higher precision and F1-score observed.

4.2.2. FNN+CNN

The top-performing models from the FNN and CNN were combined to create two distinct hybrid FNN+CNN methods, as illustrated in Figure 6. The results of the FNN+CNN approach for Method I and II are depicted in Table 5. Method I exhibited improved precision for detecting fraudulent cases compared with using FNN or CNN models alone. In Test A, the SMOTE and SMOTE+GAN showed significant improvements in precision, despite a slight decrease in recall, particularly when compared with the FNN model. This decline can be attributed to the fact that fraud predictions must meet two distinct conditions, resulting in a reduced number of predicted fraud cases. Consequently, the fraud class's precision increases, since precision is determined by the ratio of true fraud cases to predicted fraud cases. However, this trade-off leads to a decrease in the fraud class's recall. Nevertheless, the overall F1-score exhibited a slight increase compared with the individual FNN and CNN models. In Method II, different thresholds ranging from 1.1 to 1.9 were tested to determine the optimal threshold value. Since Method II's goal was to enhance recall, the best threshold value was determined based on recall.

Overall, Method II yielded better results than Method I. The findings demonstrated that the proposed hybrid FNN+CNN approach in Method II outperformed the FNN and CNN models individually. Similar to the observations on the FNN and CNN, injecting twice the number of fraud samples as the original fraud data using SMOTE and SMOTE+GAN yielded better performance than a 50:50 distribution of fraud and nonfraud samples. The performance of GAN, SMOTified-GAN, and GANified-SMOTE was not significantly affected by the number of injected fraud samples. The proposed GANified-SMOTE technique achieved the highest precision for both integration methods and also exhibited high F1-score and recall. This may be attributed to the pure variations in the FNN and CNN used in the hybrid model performing well with GANified-SMOTE. However, the GANified-SMOTE's performance on the FNN and CNN variations was similar. Therefore, it can be concluded that the proposed GANified-SMOTE can achieve high performance regardless of the number of injected fraud samples.

Table 5. The integration of FNN2+CNN2 with Method I and II was evaluated using different data generation techniques on two minority samples, and the top-performing results for each measurement in tests A and B are highlighted in bold.

Method	Generation Method	Test Samples	Threshold	PR ¹	RC ²	F1 ³
I	SMOTE	A	-	0.69	0.84	0.76
		B	-	0.88	0.86	0.87
	GAN	A	-	0.88	0.87	0.87
		B	-	0.86	0.84	0.85
	SMOTified-GAN	A	-	0.90	0.83	0.86
		B	-	0.88	0.87	0.87
	SMOTE+GAN	A	-	0.56	0.90	0.69
		B	-	0.89	0.86	0.88
	GANified-SMOTE	A	-	0.93	0.83	0.88
		B	-	0.91	0.83	0.87
II	SMOTE	A	1.1	0.51	0.86	0.64
		B	1.3	0.89	0.87	0.88
	GAN	A	1.5	0.88	0.87	0.87
		B	1.1	0.86	0.86	0.86
	SMOTified-GAN	A	1.1	0.88	0.86	0.87
		B	1.3	0.89	0.87	0.88
	SMOTE+GAN	A	1.4	0.57	0.90	0.70
		B	1.3	0.89	0.87	0.88
	GANified-SMOTE	A	1.4	0.94	0.85	0.89
		B	1.3	0.91	0.86	0.88

¹ Precision, ² Recall, ³ F1-score.

4.3. Comparison with Existing Studies

To evaluate the proposed methodologies, a comparison was made with previous studies that utilized the same dataset, as presented in Table 6. Given the trade-off between precision and recall, attaining flawless outcomes for models, whether existing or proposed, remains elusive. The outcomes observed by (Fiore et al. 2019) and (Sharma et al. 2022) upon applying SMOTE, GAN, and SMOTified-GAN exhibited relatively modest recalls (below 0.80), indicating limited detection of fraudulent transactions. Consequently, their F1-scores generally trailed behind those of the proposed methods. This serves to illustrate the proficiency of the proposed models in effectively identifying fraudulent transactions while upholding a minimal misclassification rate for nonfraudulent data.

All the implemented techniques exhibited higher recall rates compared with the existing studies. However, this improvement came at the expense of lower precision when compared with previous research. One potential explanation for this discrepancy could be the differences in the classifiers utilized. Previous studies (Fiore et al. (2019); Sharma et al. (2022)) employed classifiers with less than four layers, whereas the proposed classifier consisted of at least four. Consequently, the enhanced classifier was able to better learn the distinguishing characteristics of fraudulent data, improving the identification of such instances. However, this also led to an increased misclassification of nonfraudulent data.

Another factor that could have influenced the outcomes is the stochastic nature of the SMOTE (and the GAN) and deep learning models (the FNN). Despite the lower precision, the F1-score of the proposed methodologies surpassed that of the previous studies, except for SMOTE on Test A. This observation highlights the significant impact of classifier parameters on its performance, irrespective of the data generation methods employed. This observation aligns with the previous findings, indicating an overall enhancement in the F1-score when utilizing a hybrid of FNN and CNN, regardless of the specific data generation methods employed.

Table 6. Comparison of results obtained by existing studies and the proposed methods. Test A is the result for including twice-generated fraud samples as much as the original fraud samples, whereas Test B is the result for 50:50 fraud and nonfraud distributions. The highest Precision, Recall, and F1-score are highlighted in bold.

Generation Method	Test Samples	Classifier	PR ^a	RC ^b	F1 ^c
SMOTE (Sharma et al. 2022)	A	FNN	0.80	0.69	0.71
SMOTE	A	FNN	0.29	0.90	0.44
SMOTE (Fiore et al. 2019)	B	FNN	0.97	0.69	0.81
SMOTE	B	FNN	0.85	0.88	0.86
GAN (Sharma et al. 2022)	A	FNN	0.84	0.80	0.81
GAN	A	FNN	0.83	0.88	0.85
GAN (Fiore et al. 2019)	B	FNN	0.93	0.73	0.82
GAN	B	FNN	0.81	0.88	0.84
SMOTified-GAN (Sharma et al. 2022)	A	FNN	0.85	0.80	0.81
SMOTified-GAN	A	FNN	0.87	0.88	0.87
SMOTified-GAN	B	FNN	0.87	0.87	0.87
SMOTE+GAN	A	FNN	0.27	0.91	0.41
SMOTE+GAN	B	FNN	0.85	0.88	0.86
SMOTE+GAN	A	FNN+CNN (Method II)	0.57	0.90	0.70
SMOTE+GAN	B	FNN+CNN (Method II)	0.89	0.87	0.88
GANified-SMOTE	A	FNN	0.88	0.86	0.87
GANified-SMOTE	B	FNN	0.87	0.87	0.87
GANified-SMOTE	A	FNN+CNN (Method II)	0.94	0.85	0.89
GANified-SMOTE	B	FNN+CNN (Method II)	0.91	0.86	0.88

^a Precision, ^b Recall, ^c F1-score.

Nonetheless, data generation methods can still affect the performance of the same classifier in different variations. The results from Fiore et al. (2019) and Sharma et al. (2022) demonstrated an increase in recall and F1-score when employing GAN as opposed to SMOTE. However, in this study, the implemented GAN did not improve recall but only enhanced the F1-score on Test A. The challenges in training the GAN may have resulted in the lower quality of generated fraudulent samples. Conversely, SMOTE's random interpolation may not effectively capture the distinguishing characteristics of fraud instances. Therefore, combining SMOTE and GAN in a hybrid approach could result in the two complementing each other and better a representation of the fraudulent data. The proposed SMOTE+GAN demonstrated a slight improvement in recall on Test A compared with SMOTE. Additionally, the implemented SMOTified-GAN and the proposed GANified-SMOTE successfully improved the F1-score.

5. Conclusions

The present study introduces SMOTE+GAN and GANified-SMOTE techniques as innovative solutions to counteract class imbalance, thereby offering financial institutions an effective tool for reducing losses due to fraudulent activities. Additionally, the integration of FNN and CNN in predicting transaction categories is proposed. The effectiveness of the newly proposed data generation methods was assessed against existing techniques using an FNN, CNN, and FNN+CNN as classifiers. The outcomes highlight the potency of GANified-SMOTE, particularly when coupled with the proposed FNN+CNN classifier, in augmenting the F1-score for fraudulent data. This high F1-score indicates the method's capacity to identify a substantial portion of fraudulent transactions with reduced misclassification of legitimate transactions. Notably, GANified-SMOTE and SMOTified-GAN consistently exhibit commendable performance across varying quantities of generated minority samples. Furthermore, the research underscores the significant impact of the classifier's hyperparameter settings on classification performance, irrespective of the employed data generation methods.

In light of this experiment utilizing an online-acquired dataset, it is crucial to recognize that the study's findings may not perfectly simulate real-world scenarios marked by ever-evolving fraudulent behaviors. Future endeavors should validate the efficacy of the proposed methods within actual financial institutions. Moreover, while the experiment employs a labeled dataset with presumed accurate class labels, real-world datasets often pose the challenge of being unlabeled and necessitating comprehensive preprocessing. To tackle class labeling issues, future investigations could explore the potential of unsupervised learning in data generation. Furthermore, to firmly establish the effectiveness of the proposed methods, this study acknowledges that comparisons with existing research were limited. Factors like classifier selection may have influenced observed improvements. Therefore, to enhance generalizability, future research should involve additional classifiers and ablation studies. These efforts would serve to validate the performance of the data generation methods in diverse scenarios.

Author Contributions: Conceptualization, P.C.Y.C. and B.G.L.; methodology, P.C.Y.C.; software, P.C.Y.C.; validation, P.C.Y.C. and Y.Y.; formal analysis, P.C.Y.C.; investigation, P.C.Y.C.; resources, P.C.Y.C. and B.G.L.; data curation, P.C.Y.C.; writing—original draft preparation, P.C.Y.C.; writing—review and editing, B.G.L.; visualization, P.C.Y.C.; supervision, B.G.L.; project administration, B.G.L.; funding acquisition, B.G.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Ningbo Science and Technology Bureau grant number 2021B-008-C.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are openly available in Kaggle-Credit Card Fraud Detection, reference number [Kaggle \(2018\)](#).

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CNN	Convolutional Neural Network
FNN	Feed-forward Neural Network
SMOTE	Synthetic Minority Oversampling TEchnique
GAN	Generative Adversarial Network

References

- Abdoh, Sherif F., Mohamed Abo Rizka, and Fahima A. Maghraby. 2018. Cervical cancer diagnosis using random forest classifier with SMOTE and feature reduction techniques. *IEEE Access* 6: 59475–85. [\[CrossRef\]](#)
- Abraham, Bejoy, and Madhu S. Nair. 2018. Computer-aided diagnosis of clinically significant prostate cancer from MRI images using sparse autoencoder and random forest classifier. *Biocybernetics and Biomedical Engineering* 38: 733–44. [\[CrossRef\]](#)
- Alarfaj, Fawaz Khaled, Iqra Malik, Hikmat Ullah Khan, Naif Almusallam, Muhammad Ramzan, and Muzamil Ahmed. 2022. Credit card fraud detection using state-of-the-art machine learning and deep learning algorithms. *IEEE Access* 10: 39700–15. [\[CrossRef\]](#)
- Arafa, Ahmed, Nawal El-Fishawy, Mohammed Badawy, and Marwa Radad. 2022. RN-SMOTE: Reduced noise SMOTE based on DBSCAN for enhancing imbalanced data classification. *Journal of King Saud University—Computer and Information Sciences* 34: 5059–74. [\[CrossRef\]](#)
- Branco, Bernardo, Pedro Abreu, Ana Sofia Gomes, Mariana S. C. Almeida, João Tiago Ascensão, and Pedro Bizarro. 2020. Interleaved sequence RNNs for fraud detection. Paper presented at the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20, New York, NY, USA, August 24–27; Melbourne: Association for Computing Machinery, pp. 3101–9. [\[CrossRef\]](#)
- Bunkhumpornpat, Chumphol, Krung Sinapiromsaran, and Chidchanok Lursinsap. 2009. Safe-Level-SMOTE: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem. In *Advances in Knowledge Discovery and Data Mining*. Edited by Thanaruk Theeramunkong, Boonserm Kijsirikul, Nick Cercone and Tu-Bao Ho. Berlin and Heidelberg: Springer, pp. 475–82.
- Charitou, Charitos, Simo Dragicevic, and Artur d'Avila Garcez. 2021. Synthetic data generation for fraud detection using GANs. *arXiv* arXiv:2109.12546.

- Chawla, Nitesh V., Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* 16: 321–57. [CrossRef]
- Chen, Joy, and Kong-Long Lai. 2021. Deep convolution neural network model for credit-card fraud detection and alert. *Journal of Artificial Intelligence and Capsule Networks* 3: 101–12. [CrossRef]
- Douzas, Georgios, and Fernando Bacao. 2018. Effective data generation for imbalanced learning using conditional generative adversarial networks. *Expert Systems with Applications* 91: 464–71. [CrossRef]
- Fang, Weiwei, Xin Li, Ping Zhou, Jingwen Yan, Dazhi Jiang, and Teng Zhou. 2021. Deep learning anti-fraud model for internet loan: Where we are going. *IEEE Access* 9: 9777–84. [CrossRef]
- Fernández, Alberto, Salvador Garcia, Francisco Herrera, and Nitesh V. Chawla. 2018. SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary. *Journal of Artificial Intelligence Research* 61: 863–905. [CrossRef]
- Fiore, Ugo, Alfredo De Santis, Francesca Perla, Paolo Zanetti, and Francesco Palmieri. 2019. Using generative adversarial networks for improving classification effectiveness in credit card fraud detection. *Information Sciences* 479: 448–55. [CrossRef]
- Goodfellow, Ian, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial nets. *Advances in Neural Information Processing Systems* 27: 2672–80.
- Ileberi, Emmanuel, Yanxia Sun, and Zenghui Wang. 2021. Performance evaluation of machine learning methods for credit card fraud detection using SMOTE and AdaBoost. *IEEE Access* 9: 165286–94. [CrossRef]
- Ishaq, Abid, Saima Sadiq, Muhammad Umer, Saleem Ullah, Seyedali Mirjalili, Vaibhav Rupapara, and Michele Nappi. 2021. Improving the prediction of heart failure patients' survival using SMOTE and effective data mining techniques. *IEEE Access* 9: 39707–16. [CrossRef]
- Johnson, Justin M., and Taghi M. Khoshgoftaar. 2019. Survey on deep learning with class imbalance. *Journal of Big Data* 6: 27. [CrossRef]
- Jurgovsky, Johannes, Michael Granitzer, Konstantin Ziegler, Sylvie Calabretto, Pierre-Edouard Portier, Liyun He-Guelton, and Olivier Caelen. 2018. Sequence classification for credit-card fraud detection. *Expert Systems with Applications* 100: 234–45. [CrossRef]
- Kaggle. 2018. Credit Card Fraud Detection. Available online: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud> (accessed on 27 July 2023).
- Kim, Eunji, Jehyuk Lee, Hunsik Shin, Hoseong Yang, Sungzoon Cho, Seung kwan Nam, Youngmi Song, Jeong a Yoon, and Jong il Kim. 2019. Champion-challenger analysis for credit card fraud detection: Hybrid ensemble and deep learning. *Expert Systems with Applications* 128: 214–24. [CrossRef]
- Lan, Lan, Lei You, Zeyang Zhang, Zhiwei Fan, Weiling Zhao, Nianyin Zeng, Yidong Chen, and Xiaobo Zhou. 2020. Generative adversarial networks and its applications in biomedical informatics. *Frontiers in Public Health* 8: 164. [CrossRef]
- Mariani, Giovanni, Florian Scheidegger, Roxana Istrate, Costas Bekas, and Cristiano Malossi. 2018. BAGAN: Data augmentation with balancing GAN. *arXiv arXiv:1803.09655*. [CrossRef]
- Mirza, Shuja, Sonu Mittal, and Majid Zaman. 2018. Decision support predictive model for prognosis of diabetes using SMOTE and decision tree. *International Journal of Applied Engineering Research* 13: 9277–82.
- Mo, Zan, Yanrong Gai, and Guanlong Fan. 2019. Credit card fraud classification based on GAN-AdaBoost-DT imbalanced classification algorithm. *Journal of Computer Applications* 39: 618–22.
- Pradipta, Gede Angga, Retantyo Wardoyo, Aina Musdholifah, and I. Nyoman Hariyasa Sanjaya. 2021. Radius-SMOTE: A new oversampling technique of minority samples based on radius distance for learning from imbalanced data. *IEEE Access* 9: 74763–77. [CrossRef]
- Scott, Mitchell, and Jo Plested. 2019. GAN-SMOTE: A generative adversarial network approach to synthetic minority oversampling. *Australian Journal of Intelligent Information Processing Systems* 15: 29–35.
- Sharma, Anuraganand, Prabhat Kumar Singh, and Rohitash Chandra. 2022. SMOTified-GAN for class imbalanced pattern classification problems. *IEEE Access* 10: 30655–65. [CrossRef]
- Tahir, Muhammad, Fazlullah Khan, Mohammad Khalid Imam Rahmani, and Vinh Truong Hoang. 2020. Discrimination of golgi proteins through efficient exploitation of hybrid feature spaces coupled with SMOTE and ensemble of support vector machine. *IEEE Access* 8: 206028–38. [CrossRef]
- Xie, Yu, Guanjun Liu, Chungang Yan, Changjun Jiang, and MengChu Zhou. 2022. Time-aware attention-based gated network for credit card fraud detection by extracting transactional behaviors. *IEEE Transactions on Computational Social Systems* 10: 1004–16. [CrossRef]
- Yang, Hao, and Yun Zhou. 2021. IDA-GAN: A novel imbalanced data augmentation GAN. Paper presented at the 2020 25th International Conference on Pattern Recognition (ICPR), Milan, Italy, January 10–15; pp. 8299–305. [CrossRef]
- Ye, Huanzhuo, Lin Xiang, and Yanping Gan. 2019. Detecting financial statement fraud using random forest with SMOTE. *IOP Conference Series: Materials Science and Engineering* 612: 052051. [CrossRef]
- Zhang, Liyuan, Huamin Yang, and Zhengang Jiang. 2018. Imbalanced biomedical data classification using self-adaptive multilayer ELM combined with dynamic GAN. *Biomedical Engineering Online* 17: 181. [CrossRef] [PubMed]
- Zhang, Zhaohui, Xinxin Zhou, Xiaobo Zhang, Lizhi Wang, and Pengwei Wang. 2018. A model based on convolutional neural network for online transaction fraud detection. *Security and Communication Networks* 2018: 5680264. [CrossRef]

- Zhou, Tong, Jie Rong, Yang Liu, Weikang Gong, and Chunhua Li. 2022. An ensemble approach to predict binding hotspots in protein–RNA interactions based on SMOTE data balancing and random grouping feature selection strategies. *Bioinformatics* 38: 2452–58. [[CrossRef](#)] [[PubMed](#)]
- Zhu, Xiaoqian, Xiang Ao, Zidi Qin, Yanpeng Chang, Yang Liu, Qing He, and Jianping Li. 2021. Intelligent financial fraud detection practices in post-pandemic era. *The Innovation* 2: 100176. [[CrossRef](#)] [[PubMed](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Appendix C

Accepted Paper For ICECET



Figure C.1: ICECET Acceptance Letter

Reinforcement Learning for Optimizing Conditional Tabular Generative Adversarial Networks Hyperparameters in Financial Fraud Detection

Patience Chew Yee Cheah

School of Computer Science

University of Nottingham Ningbo China

Ningbo, China

scypc3@nottingham.edu.cn

Yue Yang

School of Computer Science

University of Nottingham Ningbo China

Ningbo, China

yue.yang2@nottingham.edu.cn

Boon Giin Lee

School of Computer Science

University of Nottingham Ningbo China

Ningbo, China

0000-0001-5743-1010

Abstract—Financial fraud datasets typically exhibit a pronounced class imbalance, leading to elevated false positive rates when employing machine learning classifiers. To mitigate this issue, the Conditional Tabular Generative Adversarial Network (CTGAN), an oversampling technique tailored for tabular data, has emerged as a promising solution by generating additional minority samples. However, it remains pertinent to investigate whether optimizing the hyperparameters governing CTGAN can yield further improvements in performance. In light of the recent application of reinforcement learning (RL) for hyperparameter tuning, this study delves into the potential of harnessing RL models to fine-tune the hyperparameters of CTGAN. The overarching objective is to design an RL-based methodology that automates the optimization of CTGAN hyperparameters. Although the initial findings from this endeavor may be inconclusive, they pave the way for future research avenues, exploring the potential of RL in the context of CTGAN hyperparameter tuning and scrutinizing the nuanced effects of CTGAN hyperparameters on classification performance.

Index Terms—class imbalance, oversampling, reinforcement learning, finance fraud detection

I. INTRODUCTION

The proliferation of digital payment systems and e-commerce platforms has given rise to an escalating occurrence of financial fraud. In response to this challenge, machine learning algorithms have been increasingly employed for the binary classification task of fraud detection, aiming to alleviate the burden of manual detection processes. However, a prevalent issue encountered in this domain is the severe class imbalance inherent in most financial fraud datasets. This imbalance entails a significant scarcity of fraudulent instances, thereby leading to a pronounced imbalance between the two classes. Consequently, machine learning models often exhibit

a propensity for biased predictions toward the majority class, consequently resulting in elevated false positive rates [1]. Consequently, the amelioration of class imbalance emerges as a critical undertaking to bolster the performance of machine learning classifiers in this context.

Numerous oversampling techniques have been proposed to rectify class imbalance by augmenting the representation of minority class samples within the dataset. The Conditional Tabular Generative Adversarial Network (CTGAN) [2] was introduced as a mechanism to generate synthetic tabular data, a format commonly encountered in financial datasets. Despite its capacity to synthesize high-quality tabular samples, thereby improving the efficacy of machine learning models, CTGAN is characterized by a suite of hyperparameters that exert a notable influence on model performance. A study conducted by [3] also revealed that the optimization of GAN-based models' hyperparameters is imperative for enhancing their performance. Hence, a pertinent inquiry arises: can the performance of machine learning models be further enhanced through the identification of optimal combinations of CTGAN hyperparameters?

Automated hyperparameter optimization techniques have been widely explored to identify the most effective hyperparameter configurations for machine learning and deep learning models across various datasets. These methodologies encompass established methods such as Grid Search [4], Random Search [5], and Bayesian Optimization [6]. Recent research has unveiled the superiority of reinforcement learning (RL)-based approaches over conventional hyperparameter optimization techniques [7], [8]. However, it is important to note that these studies primarily concentrate on optimizing the hyperparameters of machine learning or deep learning classifiers, overlooking the intricate nature of hyperparameter optimization for oversampling techniques like CTGAN. Distinct from classifier hyperparameters, evaluating the performance of oversampling methods, based solely on their generated samples, poses a challenge. The ultimate performance of

This work was supported in part by the Ningbo Science and Technology (S&T) Bureau under Grant 2021B-008-C, in part by the Ningbo Science and Technology Bureau (S&T) through the Major S&T Program under Grant 2021Z037, in part by the University of Nottingham Ningbo China (UNNC) Education Foundation through Li Dak Sum Innovation Fellowship under Grant LDS202307, and in part by a T&L Conference Grant from the UNNC Faculty of Science and Engineering.

the classifier, influenced by both the oversampling technique and classifier's hyperparameters, necessitates comprehensive consideration. Consequently, the potential utility of RL for the refinement of oversampling technique hyperparameters warrants exploration.

This study endeavors to mitigate class imbalance issues prevalent in financial datasets by investigating the feasibility of employing RL to fine-tune CTGAN hyperparameters, thereby enhancing classification performance. The objectives of this study encompass:

- 1) The proposal of a RL model tailored to the task of optimizing CTGAN hyperparameters.
- 2) A comprehensive assessment of RL model performance across various classifiers, evaluated through precision, recall, f1-score, and AUC (Area Under Curve).
- 3) An exploration of the efficacy of the proposed RL model across diverse financial fraud-related datasets.

II. LITERATURE REVIEW

Oversampling represents a data-level approach aimed at mitigating the class imbalance problem by rectifying the skewed distribution of the dataset [1]. Among the most prominent oversampling techniques, the Synthetic Minority Over-sampling Technique (SMOTE) [9] stands out, renowned for its effectiveness in addressing imbalanced classes. Its various iterations, including SMOTE+ENN, Borderline-SMOTE, ADASYN, and Safe-Level-SMOTE, have garnered widespread utilization in datasets characterized by class imbalance [10]. In recent studies, the application of Generative Adversarial Networks (GANs) [11] has gained prominence for generating minority samples. Originally conceived for the generation of realistic images, GANs have proven their mettle in surpassing SMOTE's performance when tackling class imbalance within financial fraud datasets [12]. Diverse GAN variations have been introduced to facilitate data synthesis, among which the Conditional Tabular GAN (CTGAN) [2] merits mention, specifically tailored for tabular data generation. Hybrid approaches that amalgamate SMOTE and GAN have also emerged to surmount the limitations inherent in each individual method, including SMOTified-GAN [13] and GANified-SMOTE [14].

RL [15] constitutes a paradigm wherein an agent endeavors to maximize cumulative rewards through a process of iterative decision-making based on trial-and-error learning. Typically formulated as a Markov Decision Process (MDP) [16], this framework involves the agent navigating an environment, where each state encapsulates the agent's surroundings, and each action executed by the agent engenders changes within this environment. While RL models were originally devised to excel in video game scenarios, such as Deep Q-Networks (DQN) [17], Double DQN [18], and Dueling DQN [19], which were initially devised to conquer Atari games, these techniques have transcended their gaming origins to tackle intricate real-world challenges [16]. The application of RL has been extended to address class imbalance concerns in various ways. At the algorithmic level, a DQN equipped with a reward

function that accounts for class imbalance ratios has been proposed for fraud detection [21]. At the data level, Double DQN and REINFORCE have been harnessed to facilitate the selection of training samples [22], [23]. Furthermore, the Actor-Critic methodology has been enlisted to generate minority samples [24].

RL has also exhibited noteworthy performance in the domain of hyperparameter tuning. Hyperparameter optimization can be construed as a sequential decision-making process [7], [8]. In experimental settings, RL agents featuring Long Short-Term Memory (LSTM) layers have been deployed to fine-tune neural networks [7], as well as other models like XG-Boost and Convolutional Neural Networks (CNNs) [8]. These endeavors have yielded superior outcomes when contrasted with alternative hyperparameter optimization methodologies, including Grid Search [4], Random Search [5], and Bayesian Optimization [6].

III. METHODOLOGY

This study proposes a RL approach to optimize the hyperparameters of CTGAN. The optimal CTGAN hyperparameter configuration is selected to generate minority samples, which are then combined with the original training dataset to achieve class balance. Boosting algorithms are subsequently trained on this balanced dataset to identify positive or fraudulent data instances.

A. Datasets Preprocessing

Five benchmark finance-related datasets characterized by high class imbalance were employed in this study, as detailed in Table I. These datasets were meticulously curated for machine learning applications and were devoid of missing values. Consequently, the datasets underwent simple and standard preprocessing steps, including the removal of unnecessary features (e.g., ID numbers), label encoding for categorical features, and min-max scaling for all features except the class labels. The datasets were partitioned into training (70%), validation (10%), and testing (20%) sets, preserving the original class proportions.

TABLE I
DESCRIPTION OF FIVE BENCHMARK DATASETS UTILIZED, INCLUDING THE NUMBER OF MAJORITY (MA) AND MINORITY (MI) INSTANCES

Dataset	MA Instances	MI Instances
South German Credit [25]	700	300
Africa Crisis [26]	965	94
Fake Bills [27]	971	492
Default of Credit Card Clients [28]	23364	6636
Credit Card Fraud Detection [29]	284315	492

B. RL Model For Tuning of CTGAN Hyperparameters

The hyperparameter optimization problem is conceptualized as a MDP within the RL framework. In each step, the agent observes the current hyperparameter under consideration and its corresponding value. Subsequently, the agent decides whether to increase the hyperparameter value or proceed to

TABLE II
HYPERPARAMETERS AND THEIR CORRESPONDING POSSIBLE VALUES FOR TUNING IN CTGAN

Hyperparameters	Values
embedding_dim	4, 8, 16, 32, 64, 128
generator_dim	(4,4), (8,8), (16,16), (32,32), (64,64), (128,128)
discriminator_dim	(4,4), (8,8), (16,16), (32,32), (64,64), (128,128)
generator_decay	0.0000001, 0.0000005, 0.000001, 0.000005, 0.00001
discriminator_decay	0.0000001, 0.0000005, 0.000001, 0.000005, 0.00001
generator_lr	0.000001, 0.00005, 0.0001, 0.0005, 0.001, 0.005
discriminator_lr	0.000001, 0.00005, 0.0001, 0.0005, 0.001, 0.005

the next hyperparameter. The agent progresses through all hyperparameters, leading to a final state where it receives a cumulative reward. This reward hinges on the classification performance of a machine learning classifier, which is trained on minority samples generated by CTGAN using the selected hyperparameter combinations. Importantly, the order of hyperparameter selection does not impact the RL model’s performance, as mathematically proven in prior research [8]. The components of an RL model consist of states or observations, actions, agents, and rewards.

1) *States or Observation*: In this study, each state is represented by a CTGAN hyperparameter and its respective value, as elucidated in Table III. Notably, a final state is introduced to signify the point at which the agent has selected values for all hyperparameters. To prevent agent deadlock at a particular hyperparameter, a step limit is imposed, whereby the agent transitions to the final state once this limit is reached.

2) *Actions*: To simplify the hyperparameter optimization problem, two discrete actions are employed: increasing the current hyperparameter’s value or advancing to the next hyperparameter.

3) *Agent*: Given the discrete nature of the actions, three RL agents are considered: Double Dueling DQN [17]–[19], Cross-Entropy Method (CEM) [20], and SARSA [15] as shown in Table IV. These agents operate within a discrete action space and share a common deep learning architecture featuring two hidden layers with 16 neurons each, utilizing ReLU activation for all layers except the output layer, which employs linear activation. The number of training episodes is set to 100, as preliminary studies indicate that episode rewards stabilize after initial iterations.

4) *Reward*: Defining an effective reward function is challenging, as it must guide the agent toward the ultimate objective: improving the classifier’s performance by enhancing the quality of minority samples generated by CTGAN. While standard classification metrics such as precision, recall, f1-score, and AUC are typically employed for evaluation, this study focuses on recall for the minority class and AUC in reward calculation. These metrics are deemed crucial for finance fraud detection due to their emphasis on false positive rates, a pivotal consideration in this domain. The final reward (R) aggregates improvements in AUC and recall, calculated as follows:

$$R = ((AUC_f - AUC_i) + (RC_f - RC_i)) \times 10^3 \quad (1)$$

where AUC_i and RC_i represent AUC and recall, respectively, when the classifier is trained with minority samples generated using default CTGAN hyperparameters [34] as well as AUC_f and RC_f denote AUC and recall achieved when employing the selected CTGAN hyperparameters. A scalar factor of 10^3 is applied to amplify the reward value and accentuate improvements in AUC and RC .

The agent receives the final reward upon reaching the final state. To mitigate sparse rewards, which can hinder the learning process, small rewards are issued at each step. During these intermediate steps, except for the final one, rewards are computed based on the average quality of the currently selected option. Specifically, if there is an average improvement in both recall and AUC, a reward of 1 is assigned; conversely, no improvement results in a penalty of -1. In cases where insufficient historical information exists (i.e., fewer than three previous records), no reward is provided. Additionally, to discourage the agent from fixating on a single hyperparameter, a penalty of -5 is applied if it attempts to increment a hyperparameter already at its maximum value.

C. Classifiers Training

Tree ensemble models continued to exhibit superior performance over deep learning approaches on tabular data [30]. The time efficiency of classifiers is a pivotal consideration in this study, particularly as the reinforcement learning method necessitated retraining the classifiers more than 100 times. Therefore, among the available models, three algorithms renowned for their relatively rapid execution were selected: XGBoost [31], LightGBM [32], and Histogram-based Gradient Boosting Classification Tree (HGB). These classifiers were trained using the aforementioned augmented dataset.

IV. RESULTS AND DISCUSSION

The performance evaluation of the experimented classifiers encompassed precision (PR), recall (RC), f1-score (F1) on the minority-class data, alongside the AUC. The comparative results, obtained using default CTGAN hyperparameters (as provided in the SDV library) and the fine-tuned hyperparameters (rounded to two decimal places), are meticulously tabulated in Table V. Furthermore, an analysis of performance enhancements is detailed in Table VI, wherein improvements are quantified with respect to the performance achieved under the default CTGAN hyperparameters.

The results reveal that the AUC exhibits minimal improvement, with an average enhancement close to negligible.

TABLE III
HYPERPARAMETERS AND THEIR CORRESPONDING POSSIBLE VALUES FOR TUNING IN CTGAN

Hyperparameters	Values
embedding_dim	4, 8, 16, 32, 64, 128
generator_dim	(4,4), (8,8), (16,16), (32,32), (64,64), (128,128)
discriminator_dim	(4,4), (8,8), (16,16), (32,32), (64,64), (128,128)
generator_decay	0.0000001, 0.0000005, 0.000001, 0.000005, 0.00001
discriminator_decay	0.0000001, 0.0000005, 0.000001, 0.000005, 0.00001
generator_lr	0.000001, 0.00005, 0.0001, 0.0005, 0.001, 0.005
discriminator_lr	0.000001, 0.00005, 0.0001, 0.0005, 0.001, 0.005

TABLE IV
HYPERPARAMETERS OF RL AGENTS

Agent	Hyperparameters
Double Dueling DQN	Sequential Memory
	Limit: 700
	Window length: 1
	Policy: BoltzmannQ
	Warm-up steps: 300
	Target model update: 1
	Gamma: 1.0
	Optimizer: Adam
CEM	Learning rate: 0.005
	Metrics: MAE
	Episode Parameter Memory
	Limit: 700
SARSA	Window length: 1
	Warm-up steps: 1000
	Policy: BoltzmannQ
	Warm-up steps: 300
	Gamma: 1.0
	Optimizer: Adam

The improvements in RC range between -0.07 and 0.08, signifying an absence of substantial change. Notably, PR and F1, though not directly addressed by the RL-based reward function, exhibit relatively greater improvements compared to other metrics. The maximum improvements in PR and F1 are 0.25 and 0.13, respectively. Overall, the average improvements across all metrics amount to a mere 0.01, indicating a lack of significant enhancement. This outcome does not align with the initial expectation that the RL methodology would identify optimal CTGAN hyperparameter configurations for classifier training.

A random search was conducted to evaluate the impact of CTGAN hyperparameter tuning on the overall performance of the classifiers. Out of the 194,400 potential hyperparameter combinations detailed in Table III, 50 were randomly chosen for CTGAN training and subsequent testing on the three classifiers. Models yielding the highest RC were identified, and their performance was presented in Table VII. Table VIII further examined the enhancements achieved through random search compared to default CTGAN settings. Given that selections from the random search were based on maximizing RC, they consistently outperformed default CTGAN settings. This underscores the potential for optimizing CTGAN hyperparameters to enhance its performance, resulting in an average RC improvement of approximately 4%. Conversely, the RL method failed to attain comparable RC levels.

Several factors could contribute to the suboptimal perfor-

mance of the RL-based method. These included the possibility of the RL agent struggling to effectively learn and identify optimal hyperparameters. The design of the reward function significantly influenced the agent's performance, as did the selection of hyperparameter configurations within the RL model. Moreover, the reliance of the agent on the validation dataset for reward computation introduced variability, given potential differences in data distributions between the validation and test datasets. Additionally, the stochastic nature of CTGAN may introduce variability, as identical hyperparameters may yield different sample outputs. Consequently, hyperparameter tuning might not completely alleviate noise in the generated samples.

V. CONCLUSION

This study had provided empirical insights into the performance of RL in the context of tuning CTGAN hyperparameters to enhance AUC and RC for minority class classification. The initial expectations regarding RL's effectiveness were not consistently met, as improvements in AUC and recall for the minority class did not consistently materialize following CTGAN hyperparameter tuning. Several potential factors contributing to these outcomes were explored, including potential inadequacies in the design of the RL agent's training mechanism. However, it was observed that a simple random search on CTGAN hyperparameters can achieve better performance and require less time than RL-based optimization, particularly considering the time-consuming nature of RL training, especially on large datasets. Thus, in terms of real-world feasibility, conducting a simple random search for optimal CTGAN hyperparameters appeared more practical. Nonetheless, it was essential to acknowledge that random search, by its nature, may overlook optimal hyperparameter combinations that could yield superior results. Future research efforts will concentrate on refining the RL model by exploring alternative agent architectures and investigating innovative approaches for defining the RL environment, including adaptations of reward functions, action spaces, and observations. Additionally, there was potential for exploring RL's applicability in the selection of generated synthetic data samples, offering an alternative means to enhance the performance of oversampling methods.

REFERENCES

- [1] B. Krawczyk, "Learning from imbalanced data: open challenges and future directions," *Prog. Artif. Intell.*, vol. 5, no. 4, pp. 221–232, Nov. 2016.

TABLE V
PERFORMANCE OF CLASSIFIERS BEFORE AND AFTER CTGAN HYPERPARAMETER TUNING

Dataset	Classifier	Agent	PR	RC	F1	AUC
South German Credit	HGB	Default	0.55	0.58	0.56	0.76
		DQN	0.52	0.52	0.52	0.73
		CEM	0.52	0.53	0.52	0.74
		SARSA	0.53	0.53	0.53	0.74
	LightGBM	Default	0.51	0.50	0.50	0.74
		DQN	0.52	0.58	0.55	0.75
		CEM	0.47	0.55	0.51	0.73
		SARSA	0.50	0.53	0.52	0.75
	XGBoost	Default	0.49	0.57	0.53	0.73
		DQN	0.54	0.60	0.57	0.76
		CEM	0.55	0.57	0.56	0.76
		SARSA	0.49	0.53	0.51	0.72
Africa Crisis	HGB	Default	0.95	1.00	0.97	1.00
		DQN	0.86	1.00	0.93	1.00
		CEM	0.90	1.00	0.95	1.00
		SARSA	0.90	0.95	0.92	1.00
	LightGBM	Default	0.90	0.95	0.92	1.00
		DQN	0.90	0.95	0.92	1.00
		CEM	0.95	1.00	0.97	1.00
		SARSA	0.95	1.00	0.97	1.00
	XGBoost	Default	0.95	1.00	0.97	1.00
		DQN	0.95	1.00	0.97	1.00
		CEM	0.90	1.00	0.95	1.00
		SARSA	0.95	1.00	0.97	1.00
Fake Bills	HGB	Default	0.99	0.93	0.96	1.00
		DQN	0.99	0.95	0.97	1.00
		CEM	0.99	0.92	0.95	1.00
		SARSA	0.96	0.96	0.96	1.00
	LightGBM	Default	0.97	0.95	0.96	0.99
		DQN	1.00	0.93	0.96	1.00
		CEM	1.00	0.93	0.96	1.00
		SARSA	0.97	0.96	0.96	0.99
	XGBoost	Default	0.99	0.96	0.97	0.99
		DQN	0.96	0.97	0.96	1.00
		CEM	0.99	0.95	0.97	1.00
		SARSA	0.99	0.95	0.97	1.00
Default of Credit Card Clients	HGB	Default	0.66	0.38	0.48	0.78
		DQN	0.68	0.37	0.48	0.78
		CEM	0.66	0.37	0.48	0.78
		SARSA	0.67	0.37	0.48	0.78
	LightGBM	Default	0.67	0.36	0.47	0.78
		DQN	0.68	0.37	0.48	0.78
		CEM	0.67	0.37	0.48	0.78
		SARSA	0.67	0.37	0.48	0.77
	XGBoost	Default	0.63	0.37	0.46	0.76
		DQN	0.63	0.36	0.46	0.76
		CEM	0.63	0.37	0.46	0.76
		SARSA	0.62	0.38	0.47	0.77
Credit Card Fraud Detection	HGB	Default	0.79	0.71	0.75	0.98
		DQN	0.93	0.71	0.81	0.98
		CEM	0.73	0.79	0.76	0.97
		SARSA	0.85	0.74	0.79	0.97
	LightGBM	Default	0.65	0.72	0.69	0.98
		DQN	0.90	0.73	0.81	0.97
		CEM	0.88	0.77	0.82	0.98
		SARSA	0.88	0.71	0.79	0.97
	XGBoost	Default	0.95	0.74	0.83	0.97
		DQN	0.87	0.78	0.82	0.98
		CEM	0.86	0.79	0.82	0.98
		SARSA	0.91	0.73	0.81	0.98

TABLE VI
ANALYSIS ON IMPROVEMENTS AFTER CTGAN HYPERPARAMETER
TUNING USING RL

Improvement	PR	RC	F1	AUC
Sum	0.52	0.28	0.40	0.03
Maximum	0.25	0.08	0.13	0.04
Minimum	-0.09	-0.07	-0.05	-0.03
Mean	0.01	0.01	0.01	0.00

TABLE VII
PERFORMANCE OF RANDOM SEARCH ON CTGAN

Dataset	Classifier	PR	RC	F1	AUC
South German	XGBoost	0.58	0.63	0.60	0.76
	LightGBM	0.55	0.60	0.58	0.77
	HGB	0.54	0.62	0.57	0.76
Fake Bills	XGBoost	0.99	0.97	0.98	1.00
	LightGBM	0.98	0.97	0.97	0.99
	HGB	0.98	0.97	0.97	1.00
African Crisis	XGBoost	0.95	1.00	0.97	1.00
	LightGBM	0.95	1.00	0.97	1.00
	HGB	0.95	1.00	0.97	1.00
Default of Credit Card Clients	XGBoost	0.63	0.39	0.48	0.76
	LightGBM	0.68	0.38	0.49	0.78
	HGB	0.66	0.39	0.49	0.78
Credit Card Fraud Detection	XGBoost	0.91	0.80	0.85	0.97
	LightGBM	0.71	0.82	0.76	0.98
	HGB	0.68	0.82	0.74	0.96

- [2] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling Tabular data using Conditional GAN," *Advances in Neural Information Processing Systems*, 2019.
- [3] M. Lucic, K. Kurach, M. Michalski, O. Bousquet, and S. Gelly, "Are GANs created equal? a large-scale study," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, Montréal, Canada, 2018, pp. 698–707.
- [4] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Proceedings of the 24th International Conference on Neural Information Processing Systems*, Granada, Spain, 2011, pp. 2546–2554.
- [5] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012.
- [6] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the Human Out of the Loop: A Review of Bayesian Optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2016.
- [7] H. S. Jomaa, J. Grabocka, and L. Schmidt-Thieme, "Hyp-RL: Hyperparameter Optimization by Reinforcement Learning," *ArXiv*, vol. abs/1906.11527, 2019.
- [8] J. Wu, S. Chen, and X. Liu, "Efficient hyperparameter optimization through model-based reinforcement learning," *Neurocomputing*, vol. 409, pp. 381–393, 2020.
- [9] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, Jun. 2002.
- [10] A. Fernandez, S. Garcia, F. Herrera, and N. V. Chawla, "SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary," *J. Artif. Intell. Res.*, vol. 61, pp. 863–905, Apr. 2018.
- [11] I. J. Goodfellow et al., "Generative Adversarial Networks," *arXiv [stat.ML]*, 2014.
- [12] U. Fiore, A. De Santis, F. Perla, P. Zanetti, and F. Palmieri, "Using generative adversarial networks for improving classification effectiveness in credit card fraud detection," *Information Sciences*, vol. 479, pp. 448–455, 2019.
- [13] A. Sharma, P. K. Singh, and R. Chandra, "SMOTified-GAN for Class Imbalanced Pattern Classification Problems," *IEEE Access*, vol. 10, pp. 30655–30665, 2022.
- [14] P. C. Y. Cheah, Y. Yang, and B. G. Lee, "Enhancing Financial Fraud Detection through Addressing Class Imbalance Using Hybrid SMOTE-GAN Techniques," *International Journal of Financial Studies*, vol. 11, no. 3, 2023.
- [15] R. S. Sutton and A. G. Barto, "Reinforcement learning: An introduction," The MIT Press, 2018.
- [16] F. AlMahamid and K. Grolinger, "Reinforcement Learning Algorithms: An Overview and Classification," in *2021 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*, 2021, pp. 1–7.
- [17] V. Mnih et al., "Playing Atari with Deep Reinforcement Learning," *arXiv [cs.LG]*, 2013.
- [18] H. van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-Learning," *AAAI*, pp. 2016.
- [19] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling Network Architectures for Deep Reinforcement Learning," *arXiv [cs.LG]*, 2016.
- [20] I. Szita and A. Lőrincz, "Learning Tetris Using the Noisy Cross-Entropy Method," *Neural Computation*, vol. 18, no. 12, pp. 2936–2941, 2006.
- [21] S. Vimal, K. Kayathwal, H. Wadhwa, and G. Dhama, "Application of Deep Reinforcement Learning to Payment Fraud," *arXiv [cs.LG]*, 2021.
- [22] X. Ma and W. Shi, "AESMOTE: Adversarial Reinforcement Learning With SMOTE for Anomaly Detection," *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 2, pp. 943–956, 2021.
- [23] S. Fan, X. Zhang, and Z. Song, "Imbalanced Sample Selection With Deep Reinforcement Learning for Fault Diagnosis," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 4, pp. 2518–2527, 2022.
- [24] Y. Zhou, J. Shu, X. Zhong, X. Huang, C. Luo, and J. Ai, "Oversampling Algorithm Based on Reinforcement Learning in Imbalanced Problems," in *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*, 2020, pp. 01–06.
- [25] Author(s) Initial(s). Surname(s), 2019, "South German Credit," *UCI Machine Learning Repository*, <https://doi.org/10.24432/C5X89F>
- [26] C. Reinhart, K. Rogoff, C. Trebesch, and V. Reinhart, 2019, "Africa Economic, Banking and Systemic Crisis Data," *Kaggle*, <https://www.kaggle.com/datasets/chirin/africa-economic-banking-and-systemic-crisis-data> (accessed Sep. 23, 2023).
- [27] "Fake Bills," *Kaggle*, <https://www.kaggle.com/datasets/alexandrepetit881234/fake-bills> (accessed Sep. 23, 2023).
- [28] I-C. Yeh, 2016, "Default of Credit Card Clients," *UCI Machine Learning Repository*, <https://doi.org/10.24432/C55S3H>
- [29] "Credit Card Fraud Detection," *Kaggle*, <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud> (accessed Sep. 23, 2023).
- [30] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Inf. Fusion*, vol. 81, pp. 84–90, May 2022.
- [31] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [32] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Long Beach, California, USA, 2017, pp. 3149–3157.
- [33] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [34] "CTGAN Model — SDV 0.18.0 documentation," *sdv.dev*, https://sdv.dev/SDV/user_guides/single_table/ctgan.html (accessed Sep. 23, 2023).

TABLE VIII
ANALYSIS ON IMPROVEMENT AFTER RANDOM SEARCH ON CTGAN
HYPERPARAMETERS

Improvement	PR	RC	F1	AUC
Sum	0.08	0.62	0.37	0.07
Maximum	0.08	0.10	0.08	0.04
Minimum	-0.10	0.00	0.00	-0.01
Mean	0.01	0.04	0.02	0.00

Appendix D

SYSCON Acceptance Letter

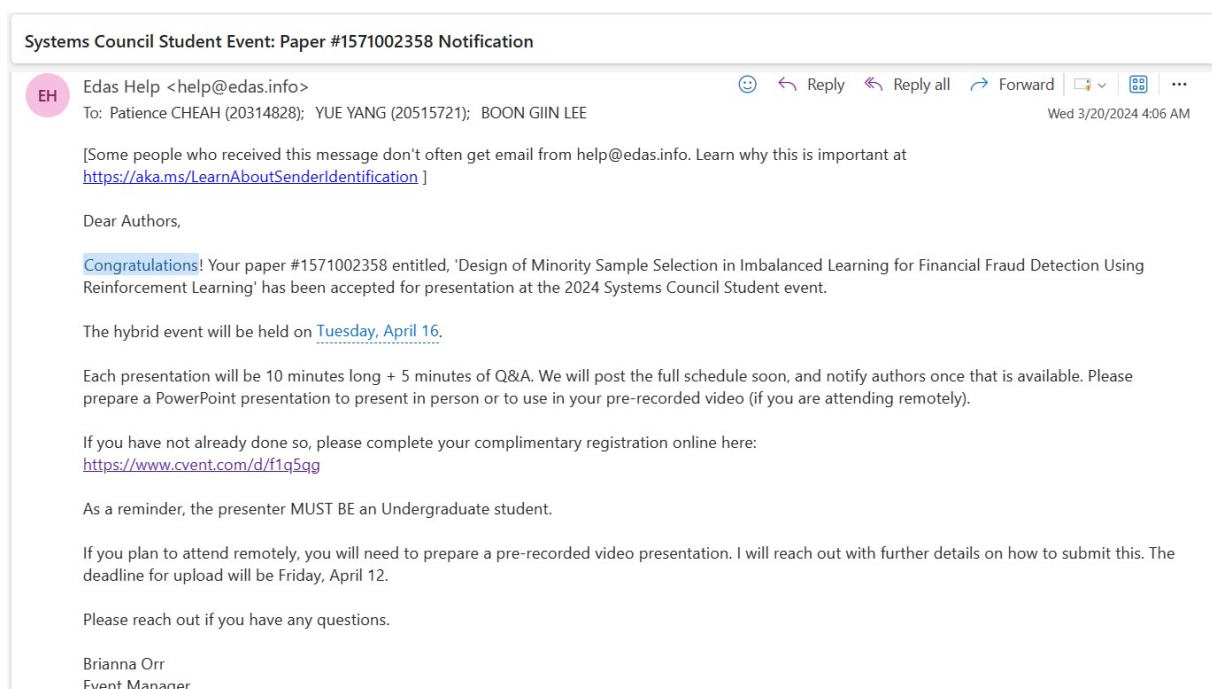


Figure D.1: SYSCON Acceptance Letter